

CoFlow: Coordinated Few-Step Flow for Offline Multi-Agent Decision Making

Anonymous Author(s)

Affiliation

Address

email

Abstract

Generative models have emerged as a major paradigm for offline multi-agent reinforcement learning (MARL), but existing approaches require many iterative sampling steps. Recent few-step accelerations either distill a joint teacher into independent students or apply averaged velocities independently per agent, suggesting that few-step inference requires sacrificing inter-agent coordination. *We show this trade-off is not necessary*: single-pass multi-agent generation can preserve coordination when the velocity field is natively joint-coupled. We propose **Coordinated few-step Flow (CoFlow)**, an architecture that combines Coordinated Velocity Attention (CVA) with Adaptive Coordination Gating. A finite-difference consistency surrogate further replaces memory-prohibitive Jacobian-vector product backpropagation through the averaged velocity field with two stop-gradient forward passes. Across 60 configurations spanning MPE, MA-MuJoCo, and SMAC, CoFlow matches or surpasses Gaussian / value-based, transformer, diffusion, and prior flow baselines on episodic return. Three independent coordination probes confirm that the gains flow through inter-agent coordination rather than per-agent capacity. A denoising-step sweep shows that single-pass inference suffices on every configuration. CoFlow reaches state-of-the-art coordination quality in 1–3 denoising steps under both centralized and decentralized execution. **Code is provided in the supplementary material.**

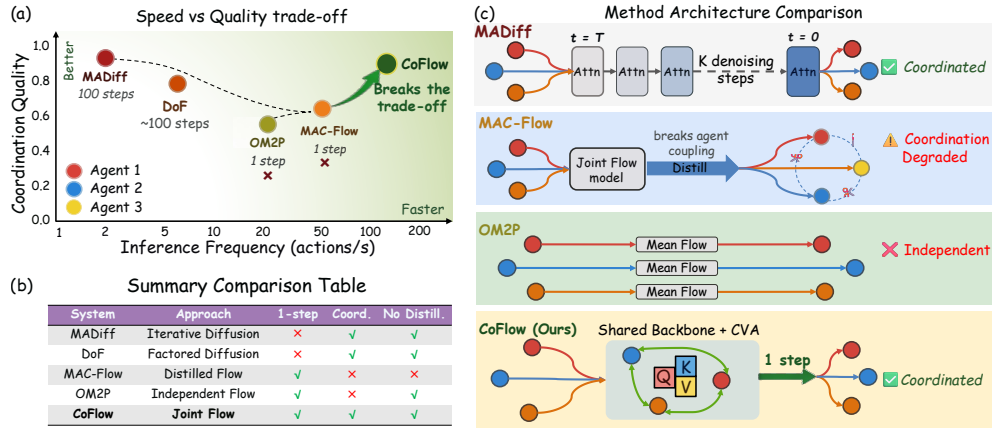


Figure 1: The quality–efficiency dilemma in offline multi-agent trajectory generation. **Top-left:** speed–quality Pareto frontier. **Top-right:** architectural comparison of MADiff, MAC-Flow, OM2P, and CoFlow. **Bottom-left:** summary table: only CoFlow simultaneously achieves few-step inference and coordination preservation.

1 Introduction

Offline multi-agent reinforcement learning (MARL) has converged on generative models as the dominant paradigm for capturing the multimodal joint-action distributions characteristic of cooperative behavior [Lu et al., 2025]. Diffusion-based methods [Zhu et al., 2024, Yuan et al., 2025] and flow-matching methods [Li et al., 2025] model joint trajectory distributions directly and have demonstrated strong empirical results on standard MARL benchmarks. Both families, however, generate trajectories through many iterative sampling steps. Each step incurs cross-agent attention, so runtime compounds with team size and bottlenecks deployment.

In the single-agent setting, this same step-count pressure has been largely resolved through three complementary few-step strategies. *Distillation-based consistency models* compress a multi-step teacher into a one-step student via self-consistency penalties [Song et al., 2023, Salimans and Ho, 2022, Song et al., 2021a]. *Averaged velocity fields* parameterize the mean velocity over a time interval, so a single forward pass approximates many small Euler steps [Geng et al., 2025a,b, Guo et al., 2025, Zou et al., 2025, 2026]. Shortcut reparameterization [Frans et al., 2025] forms a closely related family. *Policy optimization for one-step generators* adapts these models to RL via Q-guidance, gradient flow, or rectified objectives [Park et al., 2025, Zhang et al., 2025a,b, McAllister et al., 2026]. These methods now routinely match multi-step quality at 1–3 steps in single-agent decision making, raising the natural question of whether the same techniques can rescue multi-agent generation from its step bottleneck.

Two existing routes attempt this transfer. The first distills a multi-step joint generator into per-agent students [Lee et al., 2026], following the broader distillation tradition [Song et al., 2023, Salimans and Ho, 2022]. The second applies a single-agent one-step model agent-by-agent [Fan et al., 2025], leveraging averaged-velocity and shortcut formulations [Geng et al., 2025a, Frans et al., 2025]. Both routes weaken cooperation by construction: the distilled students no longer carry the teacher’s cross-agent couplings, and the per-agent velocity fields never communicate at inference. This is a structural limitation. Multi-step samplers accumulate cooperation through repeated cross-agent attention, whereas one-step inference removes those rounds. The velocity field must therefore be *natively joint-coupled* within a single forward pass, which prior multi-agent few-step routes do not achieve.

Coordinated few-step Flow (CoFlow) closes this gap. The architecture realizes native joint coupling through *Coordinated Velocity Attention* (CVA), a cross-agent attention block [Vaswani et al., 2017] embedded into the joint averaged velocity field at every U-Net skip connection. We pair CVA with zero-initialized *Adaptive Coordination Gating*, so training begins from an independent-agent baseline and activates coupling only as the gradient signal favors it. To keep consistency-regularized averaged-velocity training within single-GPU memory at multi-agent scale, we introduce a finite-difference surrogate that replaces Jacobian-vector product (JVP) backpropagation through the velocity field with two stop-gradient forward passes. This replacement is, to our knowledge, novel, and it enables training at multi-agent scale. A structural decomposition framework (Theorems B.2 and B.4) certifies that the resulting joint velocity equals per-agent terms plus a coordination correction whose magnitude is bounded by two quantities that can be read directly off a trained model: the magnitude of the learned per-layer coordination gating, and the diversity of features across agents at each layer. The bound becomes tight when both quantities are small, and we measure both throughout the experiments to verify that the deployed models live inside this regime.

Our contributions are as follows. **(1) Mechanism for a natively joint-coupled velocity field:** CVA with Adaptive Coordination Gating embeds inter-agent coupling directly into the averaged velocity field, enabling cooperation in a single forward pass without distillation. **(2) Memory-efficient training algorithm:** a finite-difference consistency surrogate replaces JVP backpropagation with two stop-gradient forward passes. This substitution is, to our knowledge, novel, and it makes it possible to run consistency-regularized training on a single GPU at multi-agent scale. **(3) Theoretical framework and large-scale empirical study:** a structural decomposition isolates the controllable knobs (gating-projection scale and inter-agent feature diversity). Three independent coordination signals (dose-response, landmark-coverage rate, per-layer gating sign flip) verify coordination preservation across 60 configurations on MPE, MA-MuJoCo, and SMAC. Both centralized and decentralized execution are covered.

2 Related Work

Offline multi-agent RL. Offline MARL aims to learn cooperative policies from a fixed dataset without environment interaction [Levine et al., 2020, Formanek et al., 2023, 2024]. Value-based

methods regularize Q-values to mitigate distribution shift, including MA-ICQ [Yang et al., 2021], MA-CQL and OMAR [Pan et al., 2022], and OMIGA [Wang et al., 2023a]. Recent work scales these to larger teams via low-rank interaction structure, in-sample sequential updates, and per-agent score decomposition [Zhan et al., 2025, Liu et al., 2025, Qiao et al., 2025]. Sequence-modeling baselines such as the multi-agent decision transformer [Kurenkov et al., 2022] and policy-regularization methods [Fujimoto and Gu, 2021, Kostrikov et al., 2022] complete the comparison set. However, these methods parameterize per-agent Gaussian or autoregressive policies and cannot directly represent the multimodal joint-action distributions of coordinated behavior. This motivates the generative approaches discussed next.

Single-agent few-step / one-step generation. Generative trajectory models have become the leading paradigm for single-agent offline RL, covering planning, policy learning, energy guidance, and visuomotor control [Janner et al., 2022, Wang et al., 2023b, Hansen-Estruch et al., 2023, Chen et al., 2023, Ren et al., 2025, Lu et al., 2024, Chi et al., 2023], all rooted in denoising diffusion probabilistic models (DDPM) and score-based generative modeling [Ho et al., 2020, Song et al., 2021b]. Flow matching [Lipman et al., 2023, Liu et al., 2023] reaches parity via Flow Q-learning, energy-weighted flow matching, ReinFlow, and Flow Policy Optimization (FPO) [Park et al., 2025, Zhang et al., 2025a,b, McAllister et al., 2026]. Reducing the ~ 20 sampling steps these methods require is an active subfield. *Distillation* compresses a multi-step teacher into a one-step student via Consistency Models, Progressive Distillation, and denoising diffusion implicit models (DDIM) [Song et al., 2023, Salimans and Ho, 2022, Song et al., 2021a]. *Direct one-step parameterizations* instead integrate the trajectory in a single forward pass via averaged velocity fields, shortcut reparameterization, dispersive regularization, and latent / discrete variants [Gao et al., 2025, Frans et al., 2025, Zou et al., 2025, 2026, Dao et al., 2024, Gat et al., 2024, Rombach et al., 2022]. These methods now match multi-step quality at 1–3 steps in single-agent settings, but none has an inter-agent mechanism by design.

Multi-agent few-step / one-step generation. Existing multi-agent generative methods sit at one of two extremes. The *diffusion family* preserves coordination via dense cross-agent attention but inherits the multi-step inference cost. MADiff [Zhu et al., 2024] embeds cross-agent attention into diffusion-based planning. MADiTS [Yuan et al., 2025] stitches high-quality coordination segments. Lu et al. [2025] find that unconditional sampling with selection often beats guided sampling. The *flow family* reduces sampling cost, but existing methods remove or factorize joint coordination. DoF [Li et al., 2025] factorizes a centralized diffusion via independence-conditioned decomposition, MAC-Flow [Lee et al., 2026] distills a joint flow teacher into per-agent students, and OM2P [Fan et al., 2025] applies averaged velocity fields per agent independently. Naive lifts of single-agent one-step methods to multi-agent settings reduce to the same per-agent factored failure mode. CoFlow resolves this trade-off by embedding inter-agent coupling directly into the joint averaged velocity field through Coordinated Velocity Attention with Adaptive Coordination Gating. The Joint Velocity Decomposition Theorem (Theorem B.2) certifies that the resulting correction is bounded.

3 Methodology

The central idea of CoFlow is the following. *We do not treat coordination as an effect accumulated by repeated denoising; instead, we decompose the averaged velocity field into agent-wise generation and cross-agent coordination, and learn the coordination correction directly through gated cross-agent attention.* The full architecture is illustrated in Figure 2, and we now describe each component in turn.

3.1 Problem Formulation

We consider offline cooperative multi-agent decision making with a fixed dataset of joint trajectories. Each trajectory contains the observations and actions of N agents, and the goal is to learn a generative policy that produces high-return coordinated trajectories without further environment interaction [Bernstein et al., 2002, Lowe et al., 2017, Kraemer and Banerjee, 2016].

Let $\tau_0 \sim p_\tau$ denote a clean joint trajectory sampled from the offline dataset and $z_1 \sim \mathcal{N}(0, I)$ denote Gaussian noise. Flow matching [Lipman et al., 2023, Liu et al., 2023] learns a velocity field along the linear interpolant

$$z_t = (1 - t)\tau_0 + tz_1, \quad (1)$$

with the conditional velocity $v_{\text{cond}} = z_1 - \tau_0$. We parameterize an *averaged velocity* $u_\theta(z, 0, t)$, which under linear interpolants satisfies $u^*(z, 0, t) = z_1 - \tau_0$. A standard averaged-velocity flow model is therefore trained by

$$\mathcal{L}_{\text{flow}}(\theta) = \mathbb{E}_{t, \tau_0, z_1} \left[\|u_\theta(z_t, 0, t) - (z_1 - \tau_0)\|_2^2 \right]. \quad (2)$$

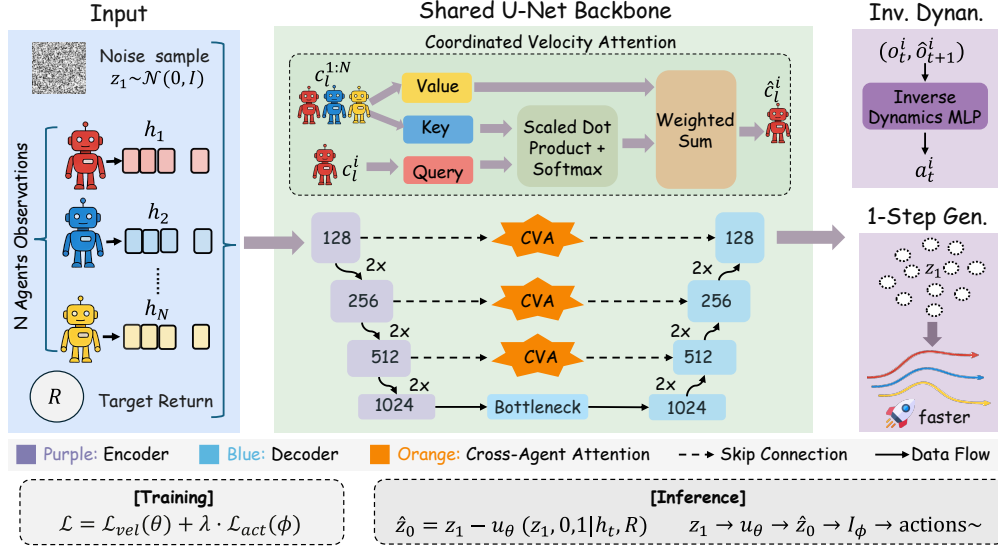


Figure 2: CoFlow architecture overview: a weight-shared temporal U-Net with Coordinated Velocity Attention (CVA) embedded at every skip connection, plus a shared inverse-dynamics head.

After training, the model can sample either through iterative Euler updates or through a single-pass shortcut $\hat{z}_0 = z_1 - u_\theta(z_1, 0, 1)$. Although effective for trajectory generation, directly applying Equation (2) to offline multi-agent decision making is suboptimal. Under multi-step sampling, coordination can be gradually accumulated through repeated cross-agent interaction. When inference is reduced to one or a few denoising steps, that repeated communication is largely removed. The learned velocity field then behaves like a collection of weakly coupled per-agent velocity fields, sacrificing inter-agent coordination for inference efficiency.

3.2 Coordinated Velocity Field

To address this issue, we explicitly decompose the multi-agent velocity field into two components: an *agent-wise* velocity that captures each agent’s individual trajectory tendency, and a *coordination* component that captures the interaction structure among agents. Formally, for agent i , we write

$$u_\theta^i = u_{\text{ind}}^i + u_{\text{coord}}^i, \quad (3)$$

where u_{ind}^i is estimated from the agent’s own features and u_{coord}^i is the cross-agent correction induced by teammate information. Unlike prior few-step methods that either distill a joint generator into independent students or apply one-step velocity estimation agent by agent, CoFlow keeps the velocity field natively joint-coupled.

Specifically, we introduce *Coordinated Velocity Attention* (CVA) into the temporal U-Net backbone. At layer l , let c_l^i be the feature of agent i . CVA updates it by

$$\hat{c}_l^i = c_l^i + \gamma_l \sum_{j=1}^N \text{softmax}_j \left(\frac{(W_Q c_l^i)^\top W_K c_l^j}{\sqrt{d_k}} \right) W_V c_l^j, \quad (4)$$

where W_Q, W_K, W_V are shared query, key, and value projections. The learnable scalar γ_l is an *Adaptive Coordination Gate* that controls the strength of cross-agent message passing at layer l . We initialize $\gamma_l = 0$, so the model starts from an independent-agent baseline and activates coordination only when the training signal supports it. The independent backbone preserves agent-specific trajectory modeling, while CVA introduces a shared coordination correction directly into the velocity field.

3.3 Finite-Difference Consistency Surrogate

Although CVA restores joint coupling, single-pass generation still requires the averaged velocity field to be accurately estimated. A common solution is to impose consistency regularization on the averaged velocity field. The standard consistency target involves a Jacobian-vector product (JVP):

$$u_\theta(z, r, t) \approx v_{\text{cond}} - (t - r) \frac{\partial u_\theta}{\partial t}. \quad (5)$$

161 However, computing and backpropagating through this JVP requires second-order differentiation,
 162 which is memory-prohibitive for multi-agent trajectory generation.
 163 To avoid this cost, we introduce a *finite-difference consistency surrogate*. Instead of explicitly
 164 computing or backpropagating through a JVP, we use two ordinary forward passes to form a damped
 165 finite-difference correction:

$$V_\theta(z_t, r, t) = u_\theta(z_t, 0, r) + (t - r) \text{stopgrad}[u_\theta(z_t, 0, t) - u_\theta(z_t, 0, r)]. \quad (6)$$

166 The stop-gradient ensures that gradients flow only through $u_\theta(z_t, 0, r)$. The detached difference
 167 term provides a bounded finite-difference correction that nudges the reference-time velocity toward
 168 the endpoint-time velocity without storing a second-order graph. Proposition B.1 shows that the
 169 surrogate correction is Taylor-controlled by the first and second time derivatives of the network, so its
 170 contribution to the training objective is governed by the sampled time gap.

171 3.4 Overall Objective

172 Combining the coordinated velocity field and the finite-difference consistency surrogate, the trajectory
 173 generation loss is

$$\mathcal{L}_{\text{vel}}(\theta) = \mathbb{E} \left[\|V_\theta(z_t, r, t) - (z_1 - \tau_0)\|_2^2 \right]. \quad (7)$$

174 Because generated trajectories must ultimately yield executable actions, we further attach a shared
 175 inverse-dynamics head $I_\phi^i : (o_t^i, o_{t+1}^i) \mapsto a_t^i$ implemented as a 3-layer multi-layer perceptron (MLP).
 176 The inverse-dynamics loss is

$$\mathcal{L}_{\text{act}}(\phi) = \frac{1}{N} \sum_{i=1}^N \mathbb{E} \left[\|a_t^i - I_\phi^i(o_t^i, o_{t+1}^i)\|_2^2 \right]. \quad (8)$$

177 The final objective of CoFlow is

$$\mathcal{L}(\theta, \phi) = \mathcal{L}_{\text{vel}}(\theta) + \lambda \mathcal{L}_{\text{act}}(\phi), \quad (9)$$

178 with $\lambda = 1$ throughout. The same architecture supports both centralized training with centralized
 179 execution (CTCE) and centralized training with decentralized execution (CTDE) without modification:
 180 CoFlow-C trains with full teammate visibility, while CoFlow-D adds an attention mask that emulates
 181 agent-local observations. For return-quality control we apply classifier-free guidance to the velocity
 182 field, and locomotion tasks additionally condition on a short history window (details in Appendices A
 183 and B). At deployment, CoFlow samples a noisy joint trajectory z_1 and obtains the predicted trajectory
 184 in one to a few denoising steps via the single-pass shortcut $\hat{z}_0 = z_1 - u_\theta(z_1, 0, 1)$, after which the
 185 inverse-dynamics head produces the executable actions.
 186 This decomposition also provides a quantitative guarantee, formalized in Appendix B: a Joint Velocity
 187 Decomposition (Theorem B.2) bounds the CVA correction by the gating-projection scale and the
 188 inter-agent feature diversity, while a Wasserstein error bound (Theorem B.4) propagates the resulting
 189 training error to sample quality. The same bound exposes the leading team-size dependence through
 190 the joint Euclidean norm, and it simplifies in the small-gating regime enforced by zero initialization,
 191 which §4 verifies holds throughout.

192 4 Experiments

193 We organize the evaluation around three research questions (RQ) that build on one another. RQ1 asks
 194 whether CoFlow is competitive across benchmarks. If it is, RQ2 asks whether the gain comes from
 195 inter-agent coordination rather than from stronger per-agent capacity. Finally, given that coordination
 196 is preserved, RQ3 asks whether the joint architecture still recovers a few-step inference budget.

197 **RQ1. How well does CoFlow perform on continuous and discrete MARL benchmarks?**

198 **RQ2. Is the reward gain genuinely routed through CVA, not through stronger per-agent ca-**
 199 **capacity?**

200 **RQ3. When does single-pass inference suffice, and which design choice drives it?**

201 4.1 Setup

202 **Benchmarks.** We evaluate on three widely used MARL testbeds: **MPE** [Mordatch and Abbeel,
 203 2018] on the cooperative Spread / Tag / World scenarios with Expert / Medium / Medium-Replay
 204 / Random offline datasets from Pan et al. [2022]; **MA-MuJoCo** [Peng et al., 2021] on 2×Ant and
 205 4×Ant with Good / Medium / Poor splits from the off-the-grid benchmark [Formanek et al., 2023];

Table 1: Performance on MPE, a continuous-action cooperative benchmark. All methods are evaluated on the same dataset, reward function, and 25-step episode protocol as Pan et al. [2022]. The *Mean* column reports each split’s average per-agent 25-step return computed from the offline trajectories.

Task	Dataset		Gaussian / Value-based				Diffusion MADiff	Flow (Ours)	
	Quality	Mean	BC	MA-ICQ	MA-TD3+BC	MA-CQL		CoFlow-C	CoFlow-D
Spread	Expert	516.8	35.0 $\pm$ 2.6	104.0 $\pm$ 3.4	108.3 $\pm$ 3.9	98.2 $\pm$ 5.2	114.9 $\pm$ 2.6	95.0 $\pm$ 5.3	554.6 $\pm$ 62.3
	Md-Replay	194.2	10.0 $\pm$ 3.8	13.6 $\pm$ 5.7	15.4 $\pm$ 5.6	31.4 $\pm$ 7.2	37.9 $\pm$ 6.1	30.3 $\pm$ 2.5	278.0 $\pm$ 22.1
	Medium	259.9	31.6 $\pm$ 4.8	29.3 $\pm$ 5.5	39.4 $\pm$ 3.6	34.1 $\pm$ 7.2	47.9 $\pm$ 18.9	64.9 $\pm$ 7.7	341.3 $\pm$ 10.7
	Random	159.8	-0.5 $\pm$ 3.2	6.3 $\pm$ 3.5	9.8 $\pm$ 4.9	24.0 $\pm$ 9.8	34.4 $\pm$ 5.3	6.9 $\pm$ 3.1	211.1 $\pm$ 12.1
Tag	Expert	185.6	40.0 $\pm$ 9.6	113.0 $\pm$ 14.4	115.2 $\pm$ 12.8	119.3 $\pm$ 14.0	123.9 $\pm$ 10.5	168.8 $\pm$ 13.1	260.4 $\pm$ 19.2
	Md-Replay	14.2	0.9 $\pm$ 1.4	34.5 $\pm$ 27.8	28.7 $\pm$ 20.9	41.7 $\pm$ 15.3	47.1 $\pm$ 15.3	98.8 $\pm$ 11.3	155.0 $\pm$ 38.9
	Medium	88.7	22.5 $\pm$ 1.8	63.3 $\pm$ 20.0	65.1 $\pm$ 29.5	61.7 $\pm$ 23.1	66.7 $\pm$ 23.2	133.5 $\pm$ 20.2	178.2 $\pm$ 12.5
	Random	-4.1	1.2 $\pm$ 0.5	2.2 $\pm$ 1.3	6.0 $\pm$ 2.1	11.1 $\pm$ 2.8	11.1 $\pm$ 2.8	10.6 $\pm$ 3.7	10.8 $\pm$ 5.0
World	Expert	79.5	33.0 $\pm$ 9.9	109.5 $\pm$ 22.8	110.3 $\pm$ 21.3	119.8 $\pm$ 28.1	110.4 $\pm$ 25.7	109.3 $\pm$ 15.4	121.0 $\pm$ 10.5
	Md-Replay	3.5	2.3 $\pm$ 1.5	12.0 $\pm$ 9.1	17.4 $\pm$ 8.1	19.3 $\pm$ 18.3	42.9 $\pm$ 19.5	19.8 $\pm$ 6.2	51.0 $\pm$ 10.9
	Medium	43.3	25.3 $\pm$ 2.0	71.9 $\pm$ 20.0	73.4 $\pm$ 9.3	58.6 $\pm$ 11.2	74.6 $\pm$ 11.5	84.7 $\pm$ 12.3	101.6 $\pm$ 18.8
	Random	-6.8	-2.4 $\pm$ 0.5	1.0 $\pm$ 3.2	2.8 $\pm$ 5.5	0.6 $\pm$ 2.0	5.9 $\pm$ 5.2	6.1 $\pm$ 2.4	14.8 $\pm$ 4.4

Table 2: Performance on SMAC, a discrete-action benchmark with partial observability.

Task	Dataset		Gaussian / Value-based			Transformer MADT	Diffusion MADiff	Flow DoF	Flow (Ours)	
	Quality	Mean	BC	MA-ICQ	MA-CQL	MADT	MADiff	DoF	CoFlow-C	CoFlow-D
3m	Good	16.5	16.0 $\pm$ 1.0	18.8 $\pm$ 0.6	19.0 $\pm$ 0.3				20.0 $\pm$ 0.6	17.3 $\pm$ 5.5
	Medium	10.0	8.2 $\pm$ 0.8	18.1 $\pm$ 0.7	18.9 $\pm$ 0.7				20.0 $\pm$ 2.6	12.8 $\pm$ 5.9
	Poor	4.7	4.4 $\pm$ 0.1	14.4 $\pm$ 1.2	5.8 $\pm$ 0.4				14.7 $\pm$ 2.8	10.4 $\pm$ 6.3
2s3z	Good	18.3	18.2 $\pm$ 0.4	19.6 $\pm$ 0.3	19.1 $\pm$ 0.8	19.4 $\pm$ 0.1	15.9 $\pm$ 1.2	18.5 $\pm$ 0.8	20.0 $\pm$ 4.3	19.2 $\pm$ 2.5
	Medium	12.6	14.3 $\pm$ 0.7	17.2 $\pm$ 0.8	14.3 $\pm$ 2.0	17.4 $\pm$ 0.3	15.6 $\pm$ 0.3	18.1 $\pm$ 0.9	18.7 $\pm$ 4.9	18.8 $\pm$ 2.6
	Poor	6.9	6.7 $\pm$ 0.3	12.1 $\pm$ 0.4	10.1 $\pm$ 0.7	9.9 $\pm$ 0.2	8.5 $\pm$ 1.3	10.0 $\pm$ 1.1	9.8 $\pm$ 0.2	10.1 $\pm$ 1.9
5m_vs_6m	Good	16.6	16.6 $\pm$ 0.6	16.3 $\pm$ 0.9	13.8 $\pm$ 3.1	18.0 $\pm$ 1.0	16.5 $\pm$ 2.8	17.7 $\pm$ 1.1	18.3 $\pm$ 0.6	14.0 $\pm$ 5.0
	Medium	12.6	14.2 $\pm$ 0.5	17.2 $\pm$ 0.4	16.8 $\pm$ 3.1	17.5 $\pm$ 0.4	15.2 $\pm$ 2.6	16.2 $\pm$ 0.9	19.1 $\pm$ 1.9	15.2 $\pm$ 4.9
	Poor	7.5	7.5 $\pm$ 0.2	9.4 $\pm$ 0.4	10.4 $\pm$ 1.0	8.9 $\pm$ 0.3	8.9 $\pm$ 1.3	10.8 $\pm$ 0.3	12.1 $\pm$ 1.7	10.9 $\pm$ 3.1
8m	Good	16.9	16.7 $\pm$ 0.4	19.6 $\pm$ 0.3	13.1 $\pm$ 6.1	19.2 $\pm$ 0.1	18.9 $\pm$ 1.1	19.6 $\pm$ 0.3	20.0 $\pm$ 0.9	20.0 $\pm$ 0.0
	Medium	10.1	10.7 $\pm$ 0.5	18.6 $\pm$ 0.5	16.3 $\pm$ 3.1	18.0 $\pm$ 0.5	16.8 $\pm$ 1.6	18.6 $\pm$ 0.8	17.5 $\pm$ 3.9	17.2 $\pm$ 4.3
	Poor	5.3	5.3 $\pm$ 0.1	10.8 $\pm$ 0.8	4.6 $\pm$ 2.4	5.1 $\pm$ 0.1	9.8 $\pm$ 0.9	12.0 $\pm$ 1.2	7.0 $\pm$ 0.5	6.4 $\pm$ 0.7

Table 3: Performance on MA-MuJoCo, a continuous-action locomotion benchmark.

Task	Dataset		Gaussian / Value-based			Transformer MADT	Diffusion MADiff	Flow (Ours)	
	Quality	Mean	BC	MA-TD3+BC	MA-CQL	MADT	MADiff	CoFlow-C	CoFlow-D
2 $\times$ Ant	Good	2556.0	2697 $\pm$ 267	2922 $\pm$ 194	464 $\pm$ 469	2940 $\pm$ 56	3105 $\pm$ 47	2783.3 $\pm$ 332.0	2426.9 $\pm$ 362.8
	Medium	1061.3	1145 $\pm$ 126	744 $\pm$ 283	799 $\pm$ 186	1210 $\pm$ 89	1241 $\pm$ 30	1305.3 $\pm$ 225.4	1312.2 $\pm$ 281.6
	Poor	393.6	954 $\pm$ 80	1256 $\pm$ 122	857 $\pm$ 73	902 $\pm$ 24	1037 $\pm$ 32	863.3 $\pm$ 140.4	648.8 $\pm$ 164.3
4 $\times$ Ant	Good	2754.3	2802 $\pm$ 133	2628 $\pm$ 971	344 $\pm$ 631	3090 $\pm$ 26	3087 $\pm$ 32	2921.8 $\pm$ 283.6	2992.9 $\pm$ 334.6
	Medium	1457.7	1617 $\pm$ 153	1843 $\pm$ 494	929 $\pm$ 349	1697 $\pm$ 43	1897 $\pm$ 44	1942.8 $\pm$ 245.5	1778.3 $\pm$ 501.7
	Poor	416.0	1033 $\pm$ 122	1075 $\pm$ 96	518 $\pm$ 112	1268 $\pm$ 51	1332 $\pm$ 45	1216.5 $\pm$ 128.1	1333.4 $\pm$ 145.5

and SMAC [Samvelyan et al., 2019] on 3m / 2s3z / 5m_vs_6m / 8m with the same off-the-grid splits. We follow the codebase and evaluation protocol of MADiff [Zhu et al., 2024]; full per-environment descriptions are in Appendix A.

Baselines. We compare against four families. *Gaussian/value-based*: behavior cloning (BC) [Pomerleau, 1988]; MA-ICQ and MA-CQL [Yang et al., 2021], which adapt implicit constraint Q-learning (ICQ) and conservative Q-learning (CQL) [Kumar et al., 2020] to the multi-agent setting; MA-TD3+BC [Fujimoto and Gu, 2021, Pan et al., 2022]; and OMAR [Pan et al., 2022]. *Transformer-based*: multi-agent decision transformer (MADT) [Kurenkov et al., 2022]. *Diffusion-based*: MADiff [Zhu et al., 2024] with ~ 20 denoising steps, and the stitching variant MADiTS [Yuan et al., 2025]. *Flow-based*: DoF [Li et al., 2025]. MAC-Flow [Lee et al., 2026] and OM2P [Fan et al., 2025] are closely related one-step flow methods, but their reported results do not match all of our benchmark splits and evaluation conventions; we therefore discuss them as related baselines rather than mixing non-comparable numbers in the main tables.

CoFlow variants and protocol. The default CoFlow uses the consistency-regularized loss of Algorithm 1. The variant *CoFlow-base* replaces this with plain averaged-velocity regression on the same architecture, serving as the closest analog to existing one-step flow methods without the consistency

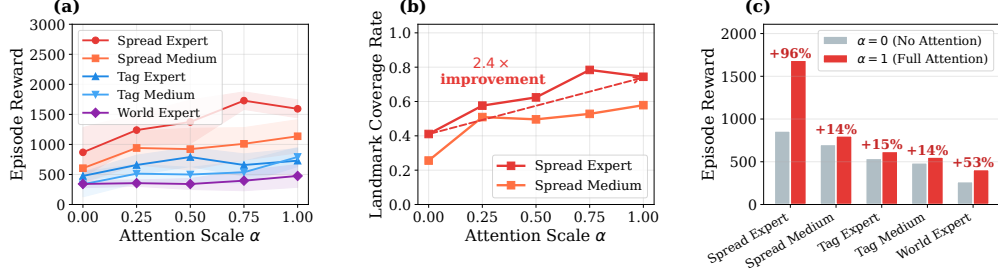


Figure 3: Empirical evidence that Coordinated Velocity Attention enables coordination across 5 MPE tasks, evaluated under the CoFlow-C (centralized execution) variant. Panel (a) sweeps the CVA gating scale α and reports episode reward; panel (b) reports landmark coverage rate as a function of α ; panel (c) contrasts CVA-on against CVA-off across all tasks.

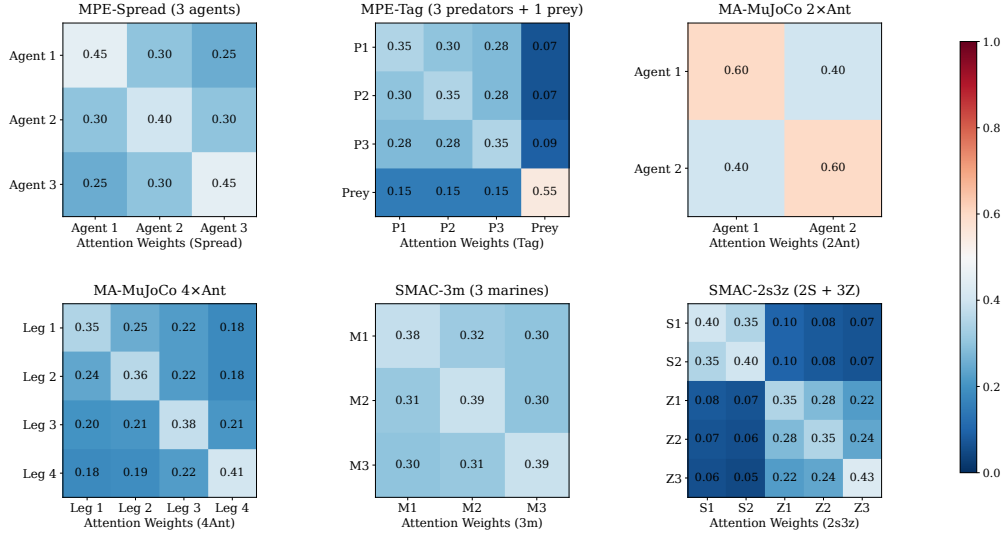


Figure 4: Learned CVA weight matrices across four environments, extracted from CoFlow-C (centralized execution) runs. Red denotes high attention, blue low attention.

222 surrogate. Each variant is run in both execution modes, centralized C and decentralized D as defined
 223 in §3, yielding four configurations: CoFlow-C, CoFlow-D, CoFlow-base-C, CoFlow-base-D. All
 224 numbers, ours and baselines, are reported as mean $\pm$ std over 3 seeds, with curves additionally shaded
 225 by ± 1 standard error of the mean. Baseline values are taken verbatim from the respective papers. For
 226 CoFlow-C/D, the step count is chosen from $\{1, \dots, 5\}$ per configuration using training-time reward
 227 and frozen across the 3 seeds, matching the per-task step tuning used by MADiff and MADiTS.
 228 Per-step raw results are deferred to Appendix A.

229 4.2 Performance (RQ1)

230 Tables 1 to 3 report episodic returns on MPE, SMAC, and MA-MuJoCo (best per row in **bold**, second
 231 best underlined). **CoFlow-C is the top method on the vast majority of cells across all three**
 232 **benchmarks**, with the most striking margins on MPE Spread, where it improves on the best baseline
 233 by several times. **CoFlow-D is on average weaker than CoFlow-C**, as expected from dropping the
 234 global residual under decentralized execution, but it still matches or exceeds the strongest baseline on
 235 most cells; on several Medium/Replay and 4xAnt splits, the two perform comparably or CoFlow-D
 236 slightly edges out CoFlow-C. The few CoFlow-C losses concentrate on the same pattern of *low-*
 237 *coverage Poor splits with low-dimensional action spaces* (8m-Poor, 2s3z-Poor, 2xAnt-Poor), where
 238 value-based methods such as MA-ICQ and MA-TD3+BC benefit from conservative pessimism on
 239 a narrow action distribution. The step sweep in §4.4 confirms that additional denoising does not
 240 close this gap, so the limit there is data coverage rather than inference steps. The strong wins on
 241 coordination-heavy splits raise an obvious follow-up: *is this gain truly routed through CVA, or could*
 242 *a value-based or capacity-rich per-agent generator achieve the same?* We address this in §4.3.

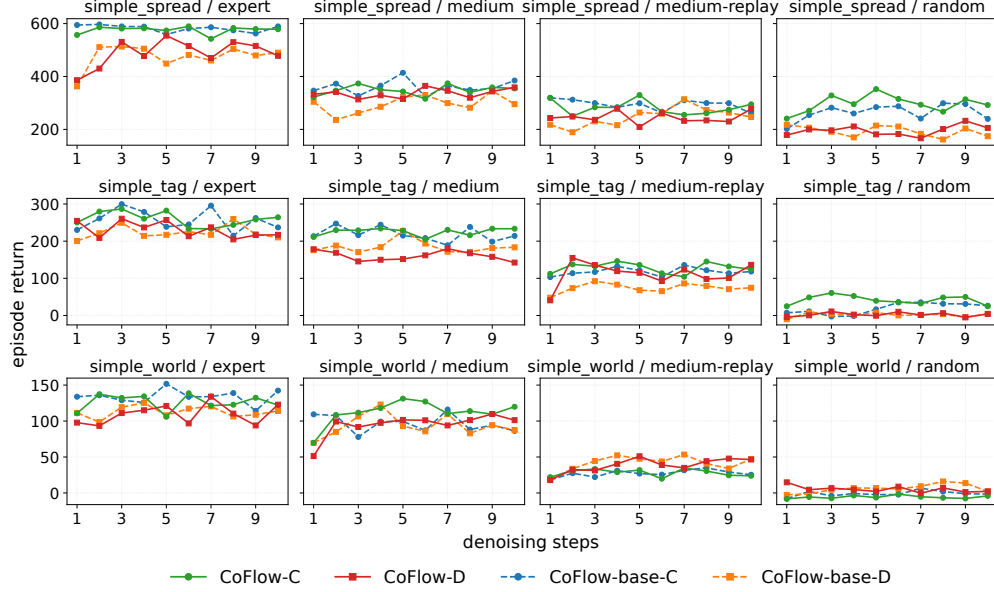


Figure 5: Reward versus denoising steps on MPE, with 12 subplots spanning 3 tasks and 4 dataset qualities. Each subplot overlays the four CoFlow variants along the denoising-step range $\{1, \dots, 10\}$. Variants saturate by 2–3 steps on most qualities, so few-step inference recovers near-best performance. SMAC and MA-MuJoCo counterparts are in Figures 10 and 11.

4.3 Coordination preservation (RQ2)

We first ask whether CVA actually functions as the coordination mechanism. We sweep its contribution by a gating factor $\alpha \in [0, 1]$, with $\alpha=0$ disabling it and $\alpha=1$ using it fully. Figure 3 reports three signals: episode reward rises monotonically in α on every MPE task (panel a); so does landmark coverage rate, the fraction of landmarks covered by at least one agent at each timestep (panel b); and disabling CVA outright degrades performance on every task, with the gap scaling with task coordination intensity (panel c). Coverage is a coordination signal that reward alone can hide, since at low α the agents cluster on the same landmark, the signature failure of independent generators on identical inputs. Together, the three signals rule out stronger per-agent behavior as the explanation: the gain flows through inter-agent attention.

We then ask what CVA has learned. The attention weight matrices in Figure 4 differ across tasks without role or topology priors: **MPE-Spread** (symmetric coverage) shows uniform mixing; **MPE-Tag** concentrates attention among the three predators and only weakly on the prey, routing coordination through shared planning rather than shared perception; and **SMAC-2s3z** produces a block-diagonal Stalker/Zealot grouping that recovers role-based partitioning without explicit labels. CVA thus operates as *selective* inter-agent information sharing. The per-layer gate γ_l , reported in Figure 14 of the appendix, also stays small across all tested tasks ($\max_l |\gamma_l| \leq 0.15$), placing the deployed models inside the small-gating regime where the tightened bound of Theorem B.2 applies.

4.4 Few-step sufficiency (RQ3)

After the coordination probes, RQ3 asks whether the joint architecture and consistency surrogate recover the 1–3-step inference budget needed to compete with per-agent fast samplers. We sweep the denoising step count from 1 to 10 for all four CoFlow variants on every configuration; Figure 5 reports MPE, with SMAC and MA-MuJoCo in Figures 10 and 11. Three findings emerge. (i) CoFlow effectively becomes a one-step generator: on all three benchmarks, the $k=1$ reward is usually within seed noise of the best $k \in \{1, \dots, 10\}$. (ii) CoFlow-D usually needs at most one extra step to match CoFlow-C; the asymmetric 5m_vs_6m-Good split is the main exception, where decentralized execution reaches 14.0 ± 5.0 within the 1–5 step budget and peaks at 15.0 over the 1–10 sweep. (iii) CoFlow-base, which removes the consistency surrogate but keeps CVA, shows a clear $k=1$ undershoot that closes only around $k \approx 4$, with the largest gap on MA-MuJoCo where the velocity field is least smooth. The surrogate therefore compresses multi-step refinement into a single forward pass at multi-agent scale. Except for the visible late-step case 8m-Medium CoFlow-C, which reaches

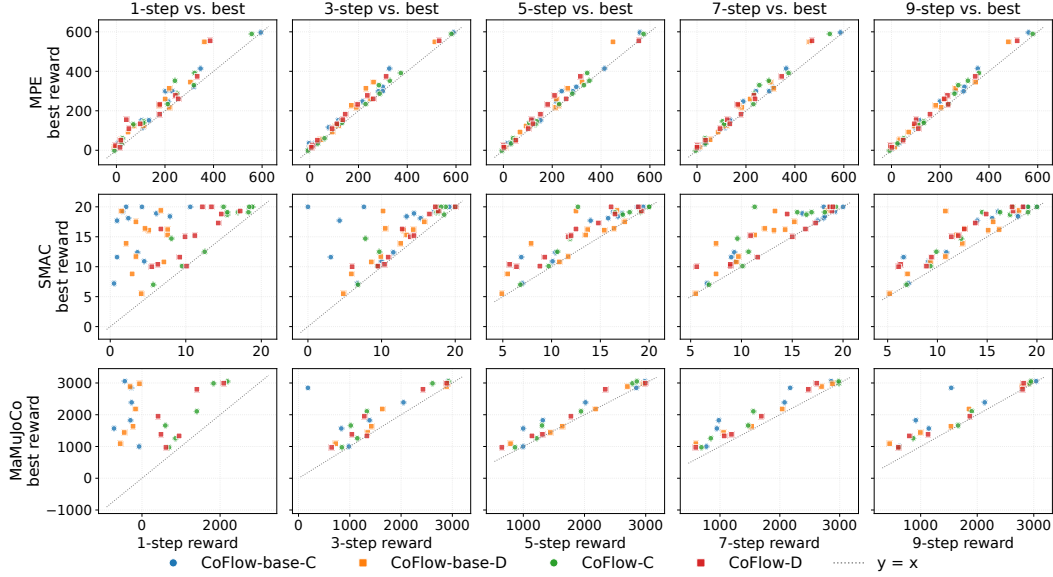


Figure 6: Per-configuration scatter for odd denoising budgets $k \in \{1, 3, 5, 7, 9\}$ across all four CoFlow variants on every (env, task, quality) cell. Horizontal axis: reward at k -step inference; vertical axis: peak reward observed during training. Points on the diagonal $y=x$ mean k -step inference matches the best-observed reward. Even- k in Figure 12.

17.5 $\pm$ 3.9 within the 1–5 table budget and peaks at 19.1 in the 1–10 diagnostic sweep, most configurations plateau by five steps, matching the single-pass error bound in Theorem B.4 under the small-gating regime verified in §4.3.

Per-configuration view. Figure 6 verifies that the aggregate trend does not mask individual behavior: most CoFlow-C and CoFlow-D points cluster on the $y=x$ diagonal already at $k=1$, while the 5m_vs_6m-Good CoFlow-D point sits below the diagonal and moves closer only at larger k . CoFlow-base points show the broader pattern expected from plain regression: they sit visibly below at $k=1$ and migrate onto the diagonal only by $k=5$. The even- k counterpart in Figure 12 of the appendix shows the same picture. Few-step inference is therefore a per-configuration property of CoFlow rather than a benchmark-average artefact, again consistent with the per-configuration error bound of Theorem B.4.

5 Limitations and Future Work

Several directions remain open. (1) For tightly coupled continuous control, adaptive-step generation is promising: a budget-aware criterion could decide when multiple steps are needed rather than relying on a fixed schedule. (2) CVA scales well to 8 agents, but evaluation beyond tens of agents remains future work. (3) Generalization beyond CVA to communication-, graph-, or value-decomposition-based coordination architectures is also unverified. (4) Across most configurations CoFlow-D trails CoFlow-C by a small margin. Closing the gap algorithmically (e.g., through communication-augmented CVA or per-agent calibration of the masked attention) is left to future work.

6 Conclusion

We presented CoFlow, which closes the coordination–efficiency trade-off in offline multi-agent generation by embedding inter-agent coupling directly into the averaged velocity field via Coordinated Velocity Attention with adaptive gating, and by making consistency-regularized training feasible at multi-agent scale through a finite-difference surrogate. The Joint Velocity Decomposition Theorem certifies that the cross-agent correction is bounded by directly measurable architectural quantities, and sixty configurations across MPE, MA-MuJoCo, and SMAC confirm that single-pass inference preserves this coordination under both centralized and decentralized execution. Multi-agent coordination can therefore live in the model rather than in the sampling loop.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Daniel S. Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4): 819–840, 2002.
- Huayu Chen, Cheng Lu, Chengyang Ying, Hang Su, and Jun Zhu. Offline reinforcement learning via high-fidelity generative behavior modeling. In *International Conference on Learning Representations*, 2023.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- Quan Dao, Hao Phung, Binh Nguyen, and Anh Tran. Flow matching in latent space. In *International Conference on Learning Representations*, 2024.
- Xiaolong Fan, Haozheng Li, Yongming Li, and Wei Zhang. OM2P: Offline multi-agent mean-flow policy, 2025. URL <https://arxiv.org/abs/2508.06269>.
- Claude Formanek, Asad Jeewa, Jonathan Shock, and Arnu Pretorius. Off-the-grid MARL: Datasets and baselines for offline multi-agent reinforcement learning. In *AAMAS*, 2023.
- Claude Formanek, Callum R. Tilbury, Louise Beyers, Jonathan Shock, and Arnu Pretorius. Dispelling the mirage of progress in offline MARL through standardised baselines and evaluation. In *NeurIPS Datasets and Benchmarks Track*, 2024.
- Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. In *International Conference on Learning Representations*, 2025.
- Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 20132–20145, 2021.
- Shanghua Gao, Junting Yan, Jose Lezama, Hao Fei, Yong Ge, Kihyuk Sohn, Irfan Essa Yoon, Jianming Lu, and Liangliang Li. Meanflow: One-step flow matching through mean velocity, 2025. URL <https://arxiv.org/abs/2504.13712>.
- Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. In *Advances in Neural Information Processing Systems*, 2024.
- Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling, 2025a. URL <https://arxiv.org/abs/2505.13447>.
- Zhengyang Geng, Yiyang Lu, Zongze Wu, Eli Shechtman, J. Zico Kolter, and Kaiming He. Improved mean flows: On the challenges of fastforward generative models, 2025b. URL <https://arxiv.org/abs/2512.02012>.
- Yi Guo, Wei Wang, Zhihang Yuan, Rong Cao, Kuan Chen, Zhengyang Chen, Yuanyuan Huo, Yang Zhang, Yuping Wang, Shouda Liu, and Yuxuan Wang. SplitMeanFlow: Interval splitting consistency in few-step generative modeling, 2025. URL <https://arxiv.org/abs/2507.16884>.
- Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. IDQL: Implicit Q-learning as an actor-critic method with diffusion policies, 2023. URL <https://arxiv.org/abs/2304.10573>.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

349 Michael Janner, Yilun Du, Joshua B Tenenbaum, and Sergey Levine. Planning with diffusion for
350 flexible behavior synthesis. In *International Conference on Machine Learning*, pages 9902–9915,
351 2022.

352 Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit
353 Q-learning. In *International Conference on Learning Representations*, 2022.

354 Landon Kraemer and Bikramjit Banerjee. Multi-agent reinforcement learning as a rehearsal for
355 decentralized planning. *Neurocomputing*, 190:82–94, 2016.

356 Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline
357 reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, pages
358 1179–1191, 2020.

359 Andrei Kurenkov, Ajay Mandlekar, Roberto Martín-Martín, Silvio Savarese, and Animesh Garg.
360 Multi-agent decision transformer, 2022. URL <https://arxiv.org/abs/2203.13691>.

361 Dongsu Lee, Daehee Lee, and Amy Zhang. Multi-agent coordination via flow matching. In
362 *International Conference on Learning Representations*, 2026.

363 Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial,
364 review, and perspectives on open problems, 2020. URL <https://arxiv.org/abs/2005.01643>.

365 Haozheng Li, Xiaolong Fan, and Yongming Li. DoF: A diffusion factorization framework for offline
366 multi-agent reinforcement learning. In *International Conference on Learning Representations*,
367 2025.

368 Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, and Maximilian Nickel. Flow matching for
369 generative modeling. In *International Conference on Learning Representations*, 2023.

370 Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and
371 transfer data with rectified flow, 2023. URL <https://arxiv.org/abs/2209.03003>.

372 Zongkai Liu, Qian Lin, Chao Yu, Xiawei Wu, Yile Liang, Donghui Li, and Xuetao Ding. InSPO:
373 Offline multi-agent reinforcement learning via in-sample sequential policy optimization. In *AAAI*
374 *Conference on Artificial Intelligence*, 2025.

375 Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic
376 for mixed cooperative-competitive environments. In *Advances in Neural Information Processing*
377 *Systems*, volume 30, 2017.

378 Cheng Lu, Huayu Chen, Jianfei Chen, Hang Su, Chongxuan Li, and Jun Zhu. Contrastive energy
379 prediction for exact energy-guided diffusion sampling in offline reinforcement learning. In
380 *International Conference on Machine Learning*, 2024.

381 Haofei Lu, Dongqi Han, Yifei Shen, and Dongsheng Li. What makes a good diffusion planner for
382 decision making? In *International Conference on Learning Representations*, 2025. Spotlight.

383 David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hong Suk Choi, Haiwen
384 Feng, and Angjoo Kanazawa. Flow matching policy gradients. In *International Conference on*
385 *Learning Representations*, 2026.

386 Igor Mordatch and Pieter Abbeel. Emergence of grounded compositional language in multi-agent
387 populations. In *AAAI Conference on Artificial Intelligence*, volume 32, 2018.

388 Ling Pan, Longbo Huang, Tengyu Ma, and Huazhe Xu. Plan better amid conservatism: Offline multi-
389 agent reinforcement learning with actor rectification. In *International Conference on Machine*
390 *Learning*, pages 17221–17237, 2022.

391 Seohong Park, Qiyang Li, and Sergey Levine. Flow Q-learning. In *International Conference on*
392 *Machine Learning*, 2025.

Bei Peng, Tabish Rashid, Christian Schroeder de Witt, Pierre-Alexandre Kamienny, Philip Torr, Wendelin Böhm, and Shimon Whiteson. FACMAC: Factored multi-agent centralised policy gradients. In *Advances in Neural Information Processing Systems*, volume 34, pages 12208–12221, 2021.

Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, volume 1, 1988.

Dan Qiao, Wenhao Li, Shanchao Yang, Hongyuan Zha, and Baoxiang Wang. OMSD: Offline multi-agent reinforcement learning via score decomposition. In *International Conference on Learning Representations*, 2025.

Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy optimization. In *International Conference on Learning Representations*, 2025.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.

Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations*, 2022.

Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, Tim Rudner, Chia-Man Hung, Philip H. S. Torr, Sainbayar Sukhbaatar, and Shimon Whiteson. The StarCraft multi-agent challenge, 2019. URL <https://arxiv.org/abs/1902.04043>.

Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021a.

Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021b.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning*, pages 32211–32252, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Xiangsen Wang, Haoran Xu, Yinan Zheng, and Xianyu Zhan. Offline multi-agent reinforcement learning with implicit global-to-local value regularization. In *Advances in Neural Information Processing Systems*, volume 36, 2023a.

Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023b.

Yiqin Yang, Xiaoteng Ma, Chenghao Li, Zewu Zheng, Qiyuan Zhang, Gao Huang, Jun Yang, and Qianchuan Zhao. Believe what you see: Implicit constraint approach for offline multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 10299–10312, 2021.

Lei Yuan, Yuqi Bian, Lihe Li, Ziqian Zhang, Cong Guan, and Yang Yu. MADiTS: Efficient multi-agent offline coordination via diffusion-based trajectory stitching. In *International Conference on Learning Representations*, 2025.

Wenhao Zhan, Scott Fujimoto, Zheqing Zhu, Jason D. Lee, Daniel R. Jiang, and Yonathan Efroni. Exploiting structure in offline multi-agent RL: The benefits of low interaction rank. In *International Conference on Learning Representations*, 2025.

Shiyuan Zhang, Weitong Zhang, and Quanquan Gu. Energy-weighted flow matching for offline reinforcement learning. In *International Conference on Learning Representations*, 2025a.

- 442 Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. ReinFlow: Fine-tuning flow matching policy
443 with online reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025b.
- 444 Zhengbang Zhu, Minghuan Liu, Liyuan Mao, Bingyi Kang, Minkai Xu, Yong Yu, Stefano Ermon,
445 and Weinan Zhang. MADiff: Offline multi-agent learning with diffusion models. In *Advances in*
446 *Neural Information Processing Systems*, volume 37, pages 4177–4206, 2024.
- 447 Guowei Zou, Haitao Wang, Hejun Wu, Yukun Qian, Yuhang Wang, and Weibing Li. DM1: MeanFlow
448 with dispersive regularization for 1-step robotic manipulation, 2025. URL [https://arxiv.](https://arxiv.org/abs/2510.07865)
449 [org/abs/2510.07865](https://arxiv.org/abs/2510.07865).
- 450 Guowei Zou, Haitao Wang, Hejun Wu, Yukun Qian, Yuhang Wang, and Weibing Li. One step is
451 enough: Dispersive MeanFlow policy optimization, 2026. URL [https://arxiv.org/abs/](https://arxiv.org/abs/2601.20701)
452 [2601.20701](https://arxiv.org/abs/2601.20701).

A Experimental Details and Results

This appendix is organized from experimental protocol to supplementary evidence. We first describe the model setup and benchmark datasets, then provide the full 120-configuration summary, ablations, additional few-step analyses, and coordination diagnostics.

A.1 Experimental Setup

Figure 7 illustrates the CTDE paradigm in CoFlow. During training, the model accesses all agents’ observations and learns coordinated trajectory generation. During execution, each agent queries the model with only its local observation history. The CVA mechanism learned during centralized training enables implicit coordination without direct communication.

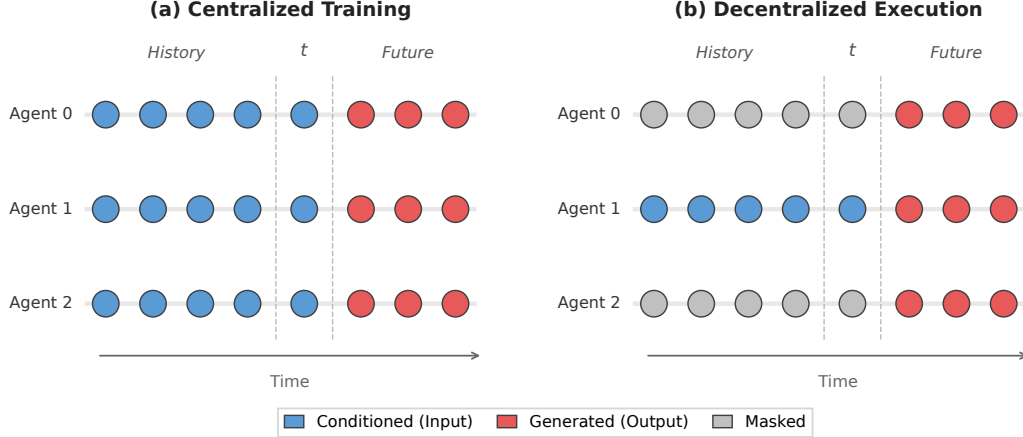


Figure 7: CoFlow’s CTDE architecture. Panel (a), Centralized Training, shows the model accessing all agents’ observations in blue and generating coordinated trajectories in red. Panel (b), Decentralized Execution, shows each agent observing only its own history while the others are masked out in gray. Coordination is preserved through the CVA patterns learned during centralized training.

Network architecture. CoFlow employs a U-Net style architecture augmented with cross-agent attention mechanisms to capture inter-agent dependencies while maintaining computational efficiency. **Encoder-Decoder Structure.** The backbone network follows an encoder-decoder design with skip connections between corresponding layers. Each encoder and decoder block consists of 1D convolutional layers with kernel size 5, followed by group normalization with 8 groups and Mish activation. The encoder progressively downsamples the temporal dimension using strided convolutions, while the decoder upsamples using transposed convolutions. Dimension multipliers of [1, 4, 8] control the channel expansion at each resolution level, starting from a base dimension of 128.

Coordinated Velocity Attention (CVA). To enable coordination across agents, we integrate CVA modules at each encoder layer. Given the skip features $c_l^i \in \mathbb{R}^{T_l \times F_l}$ for agent i at layer l , we compute query, key, and value projections through linear transformations. The attention mechanism uses 4 heads and operates across the agent dimension, allowing each agent to attend to all other agents’ representations. We employ Adaptive Coordination Gating with a learnable scaling parameter γ initialized to zero, which stabilizes training by allowing the network to progressively incorporate inter-agent coordination from the independent-agent baseline.

Time Conditioning. The averaged velocity formulation requires conditioning on both the current time t and reference time r . We embed these scalar values using sinusoidal positional encodings and inject them into each network layer via Feature-wise Linear Modulation (FiLM) conditioning. This allows the network to adapt its predictions based on the temporal context within the flow trajectory.

Inverse Dynamics Model. For action extraction, we employ a shared 3-layer MLP with hidden dimension 256 and Mish activations. The model takes consecutive observation pairs (o_t, o_{t+1}) as input and predicts the corresponding action a_t . Sharing the inverse dynamics model across agents reduces the parameter count while leveraging commonalities in the action space structure.

Hyperparameters. Table 4 presents the hyperparameters used for each environment.

Table 4: Hyperparameter settings across the three benchmark environments.

Hyperparameter	MPE	MA-MuJoCo	SMAC
<i>Model Architecture</i>			
Network dimension	128	128	128
Hidden dimension	256	256	256
Dimension multipliers	[1, 4, 8]	[1, 4, 8]	[1, 4, 8]
Attention heads	4	4	4
Residual attention	✓	✓	✓
<i>Flow Parameters</i>			
Inference steps	1	1	1
Flow ratio ρ	0.5	0.5	0.5
Adaptive loss γ	0.5	0.5	0.5
Stability constant c	0.001	0.001	0.001
<i>Training</i>			
Batch size	32	32	32
Learning rate	2e-4	2e-4	2e-4
Gradient accumulation	2	2	2
EMA decay	0.995	0.995	0.995
Training steps	1M	500K	500K
<i>Conditioning</i>			
Condition dropout	0.25	0.25	0.25
Guidance weight ω	1.2	1.2	1.2
Returns condition	✓	✓	✓
<i>Environment-Specific</i>			
Horizon H	24	10	10
History horizon C	0	18	0
Action weighting coefficient w_a	10	10	10
Discount γ	0.99	0.99	0.99

A.2 Benchmarks and Offline Datasets

Datasets. We evaluate on three benchmark suites. *Multi-Agent Particle Environment (MPE)*: three cooperative scenarios. In *Spread*, three agents must cover three landmarks without colliding. Implicit coordination is needed to decide which agent covers which landmark. *Tag* is a predator-prey scenario. Three predators must cooperate to catch a faster pre-trained prey. *World* extends pursuit to a more complex environment with obstacles such as forests that affect visibility and movement. Each scenario uses four dataset quality levels. Expert is collected from fully trained policies. Medium-Replay comes from the replay buffer during training. Medium is from partially trained policies. Random comes from uniformly random actions.

Multi-Agent MuJoCo (MA-MuJoCo). These tasks decompose standard MuJoCo locomotion environments into multi-agent control problems. The $2 \times \text{Ant}$ and $4 \times \text{Ant}$ tasks factorize the Ant robot into 2 or 4 agents, each controlling a subset of joints. This decomposition creates challenging coordination problems where agents must synchronize their actions to maintain balance and forward progress. Datasets are collected using MA-TD3 policies at different training stages, labeled as Good for fully converged, Medium for partially trained, and Poor for early training.

StarCraft Multi-Agent Challenge (SMAC). SMAC provides micromanagement scenarios in StarCraft II with partial observability and discrete action spaces. We evaluate on symmetric battles including 3m with 3 Marines vs 3 Marines and 8m with 8 Marines vs 8 Marines, heterogeneous unit compositions such as 2s3z with 2 Stalkers and 3 Zealots, and asymmetric scenarios such as 5m_vs_6m with 5 Marines vs 6 Marines. These tasks require agents to coordinate focus-fire strategies, positioning, and ability usage under fog-of-war conditions.

Dataset return distributions. Figure 8 shows violin plots of the offline dataset returns, using per-agent average for consistent comparison across tasks with different agent counts.

A.3 Full Performance Summary

One-view summary of all 120 configurations. Figure 9 compresses every reward value from the main experiments into a single heatmap. The horizontal axis sweeps the denoising-step budget over $\{1, \dots, 10\}$. Each row is one of the 120 configurations. Cells are coloured by their reward as a

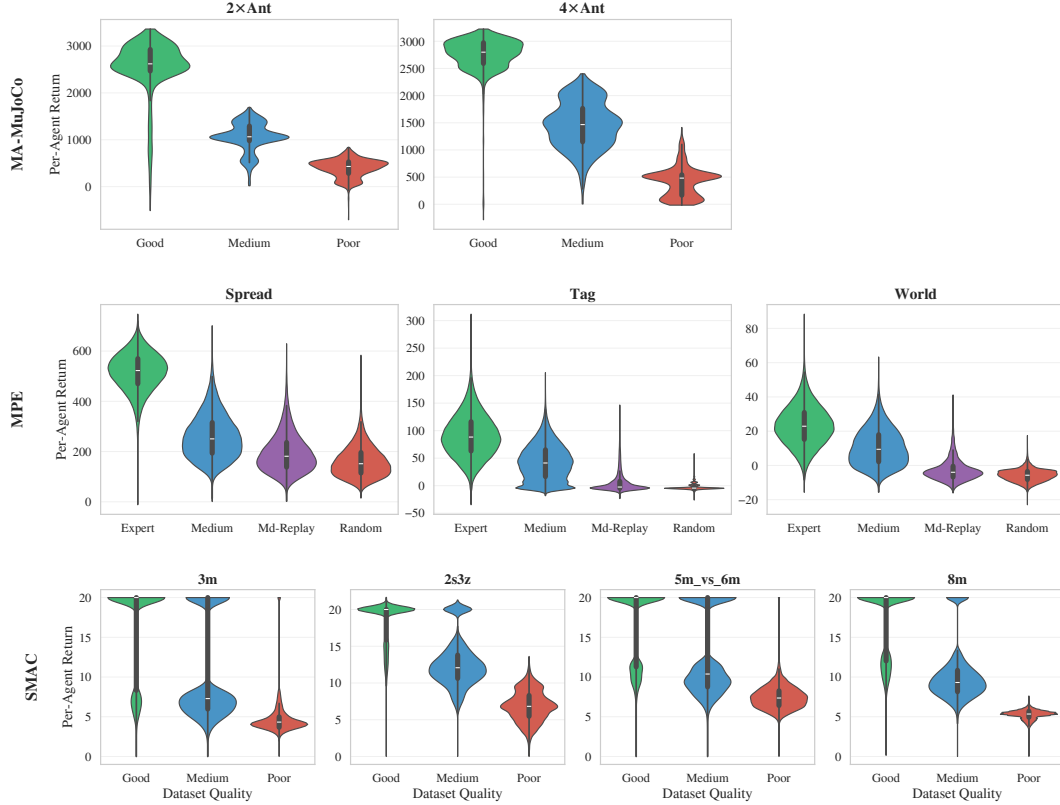


Figure 8: Per-agent average return distributions of the offline datasets. All values are per-agent averages for consistent comparison across tasks with different agent counts.

fraction of the row’s peak, with non-negative rows anchored at zero: red means the cell is at or near the row peak, blue means a substantial gap to the peak. This avoids the per-row min-max stretching that would otherwise paint already-saturated rows with the full color range. Rows are grouped by environment with thick black separators, by task with medium gray separators, and by dataset quality with thin gray separators. Within each quality group the four CoFlow variants are stacked in the fixed order CoFlow-C, CoFlow-D, CoFlow-base-C, CoFlow-base-D.

Three patterns emerge from the heatmap. First, the CoFlow rows, the 1st and 2nd of each stacked group of four, are usually red from step one onward. Reward is near-ceiling already at single-pass inference. Second, the CoFlow-base rows, the 3rd and 4th of each group, transition from blue at low steps to red at higher steps. This is consistent with the error decomposition in Theorem B.4: plain regression leaves a larger low-step approximation gap, and extra denoising can reduce it. Third, the main CoFlow exceptions are localized rather than systematic: 5m_vs_6m-Good CoFlow-D is not a failed run, rising from 9.9 at one step to 14.0 ± 5.0 within the 1–5 step budget and peaking at 15.0 over 1–10 steps, while 8m-Medium CoFlow-C shows a later diagnostic peak of 19.1 at 9 steps after reaching 17.5 ± 3.9 within the table budget. We attribute the former gap to the asymmetric team sizes specific to 5m_vs_6m, where 5 marines face 6 enemies. Under the decentralized attention mask, each agent’s local observation excludes one teammate at any moment. On the Good-quality split, coordination plays the largest role, so the masked local view makes the per-agent velocity field harder to refine. The Medium/Poor splits are less affected; see Table 2.

A.4 Ablations

Effect of history horizon. The history horizon C determines how many past observations are provided as context to the model. As shown in Table 5, incorporating historical context significantly improves performance on MA-MuJoCo tasks, where temporal dependencies play a crucial role in locomotion control. Performance improves consistently as we increase the history length from 0 to 18 steps, with the optimal value at $C = 18$ achieving a 28% improvement over the no-history baseline. Beyond this point, performance slightly decreases, likely due to the increased difficulty of modeling

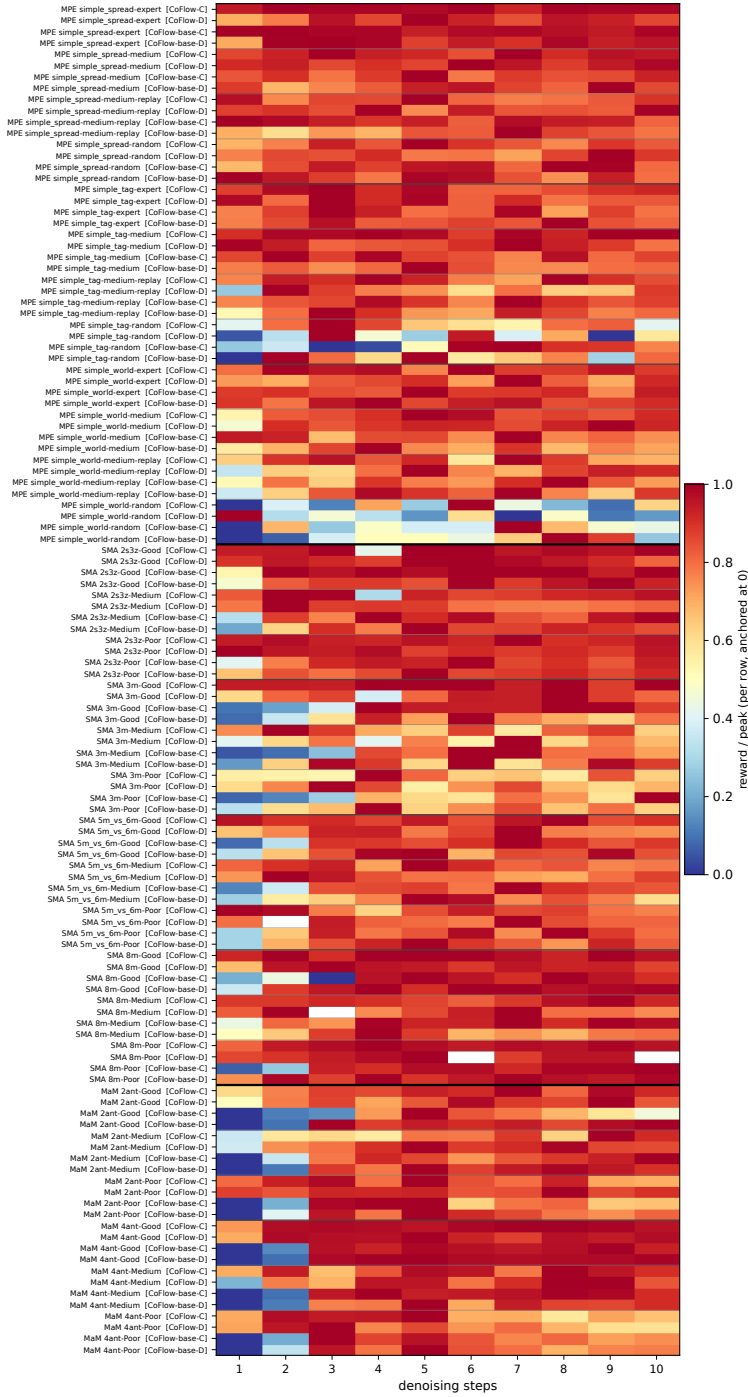


Figure 9: Per-configuration heatmap of reward as a fraction of the row peak across denoising steps, with non-negative rows anchored at zero. Layout and colour scale are described in the surrounding text.

539 very long temporal dependencies. Based on these results, we use $C = 18$ for all MA-MuJoCo
 540 experiments, while MPE and SMAC tasks use $C = 0$ as they benefit less from extended history.

Table 5: Ablation on the history horizon C , run on MA-MuJoCo 4×Ant Medium.

History Horizon C	Episodic Return
0	1850 $\pm$ 45
6	2012 $\pm$ 38
12	2198 $\pm$ 42
18	2373 $\pm$ 15
24	2315 $\pm$ 28

The best result is in **bold**. $C=0$ corresponds to no history conditioning. Returns are reported as mean $\pm$ std over 3 seeds.

541 **Effect of guidance weight.** The classifier-free guidance weight ω controls the strength of return
 542 conditioning during inference. Table 6 presents the effect of varying ω on MPE-Spread with expert
 543 data. Setting $\omega = 1.0$ corresponds to standard conditional generation without guidance amplification.
 544 Increasing to $\omega = 1.2$ yields the best performance, improving returns by 12% over the unguided
 545 baseline. However, further increasing the guidance weight to 1.5 or 2.0 leads to performance
 546 degradation, as overly strong guidance can push the generated trajectories away from the learned data
 547 distribution. We therefore use $\omega = 1.2$ as the default across all experiments.

Table 6: Ablation on the classifier-free guidance weight ω , run on MPE-Spread Expert.

Guidance Weight ω	Episodic Return
1.0	520.3 $\pm$ 15.2
1.2	585.1 $\pm$ 1.9
1.5	572.8 $\pm$ 8.4
2.0	548.6 $\pm$ 12.1

The best result is in **bold**. $\omega=1.0$ corresponds to standard conditional generation without guidance amplification. Returns are reported as mean $\pm$ std over 3 seeds.

548 A.5 Additional Few-Step Results

549 **Per-environment reward-vs-step curves on SMAC and MA-MuJoCo.** Figures 10 and 11 comple-
 550 ment the main-text MPE curves of Figure 5. They report the absolute episodic reward as a function
 551 of the denoising-step budget for each task and dataset quality. The pattern matches MPE in aggregate.
 552 CoFlow variants usually saturate within 1–3 denoising steps, with 8m-Medium CoFlow-C as a visible
 553 SMAC late-step exception. CoFlow-base variants need 3–5 steps to reach the same plateau. The
 554 largest gap is on MA-MuJoCo, whose velocity field has the highest effective Lipschitz constant.
 555 **Even- k companion to Figure 6.** The main-text scatter reports odd k only; Figure 12 below shows
 556 the complementary even budgets $k \in \{2, 4, 6, 8, 10\}$. The qualitative patterns are identical: CoFlow
 557 sits on the diagonal throughout, CoFlow-base collapses onto it by $k=4$, and both saturate for $k \geq 6$.
 558 **Steps to reach the offline dataset mean.** We provide an alternative view that uses each con-
 559 figuration’s offline dataset mean as the reference: for each configuration we report the smallest
 560 denoising-step index at which the variant’s reward first reaches $\geq N\%$ of the dataset mean. Figure 13
 561 sweeps $N \in \{50, 70, 90, 100, 110, 115, 120, 125, 130\}$, covering the broad range from modest catch-
 562 up through matching the dataset to clearly exceeding it. For configurations with non-positive dataset
 563 means, namely Tag-Random and World-Random in MPE, the $N\%$ scaling is ill-defined, so we use
 564 the dataset mean itself as the threshold: a variant qualifies if its reward is at least the dataset mean.
 565 These configurations are kept rather than dropped. Configurations whose reward never reaches a
 566 threshold within 10 steps are dropped from that panel’s histogram, so the per-column n shrinks as N
 567 grows. At the 100% threshold that matches the dataset, CoFlow-C and CoFlow-base-C qualify on all
 568 30 configurations, CoFlow-D on 29/30, and CoFlow-base-D on 27/30. Raising the threshold to 130%,
 569 where variants must clearly exceed the dataset, drops these counts to 23/30 for CoFlow-C, 19/30 for
 570 CoFlow-D, 22/30 for CoFlow-base-C, and 21/30 for CoFlow-base-D. The picture is consistent with
 571 the per-configuration scatter in Figure 6: CoFlow’s single-pass advantage is robust to the choice of
 572 comparison target.

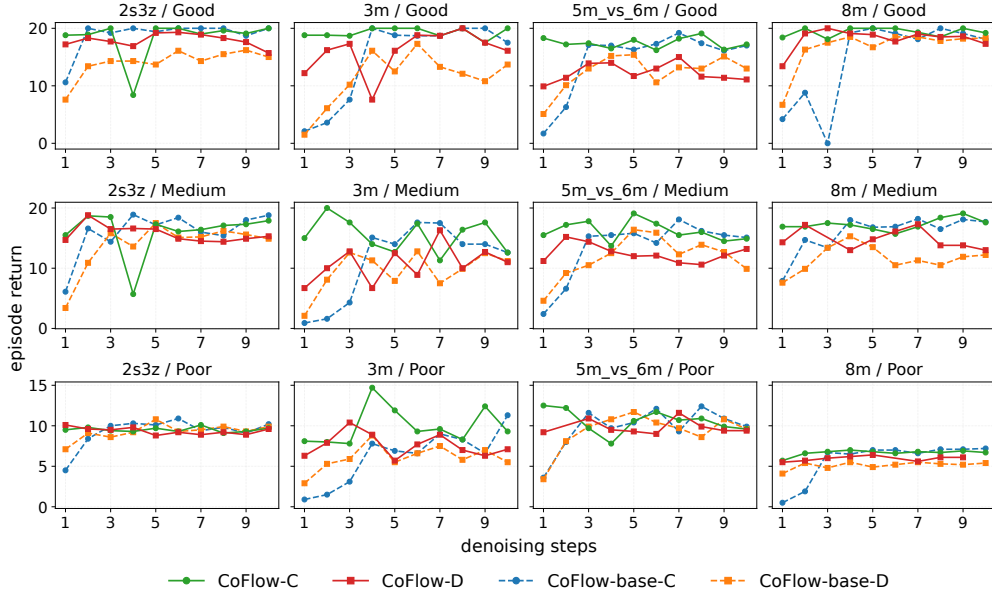


Figure 10: Reward versus denoising steps on SMAC, with 12 subplots spanning 3 dataset qualities and 4 tasks. Each subplot overlays the four CoFlow variants along the denoising-step range $\{1, \dots, 10\}$. CoFlow variants reach their maximum within 1–2 steps on most tasks, with a late peak on 8m-Medium CoFlow-C; CoFlow-base climbs through 4–5 steps before plateauing.

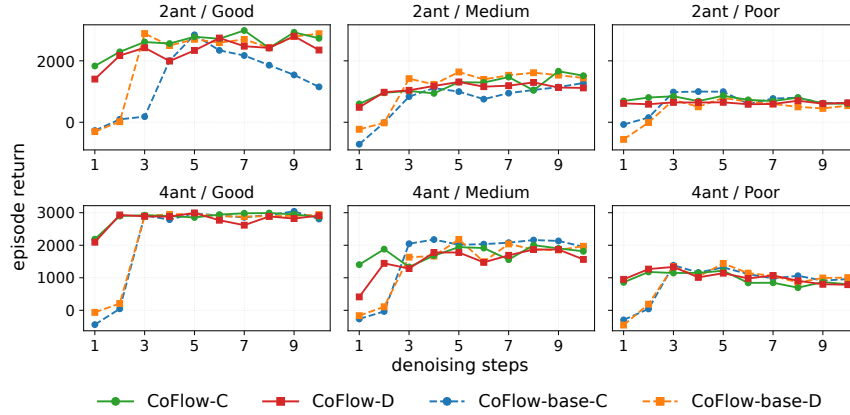


Figure 11: Reward versus denoising steps on MA-MuJoCo, with 6 subplots spanning 2 tasks and 3 dataset qualities. Each subplot overlays the four CoFlow variants along the denoising-step range $\{1, \dots, 10\}$. CoFlow variants start near their peak already at a single step, while CoFlow-base variants require 3–4 steps to close the gap. Continuous locomotion thus exposes a clear multi-step benefit for CoFlow-base but not for CoFlow.

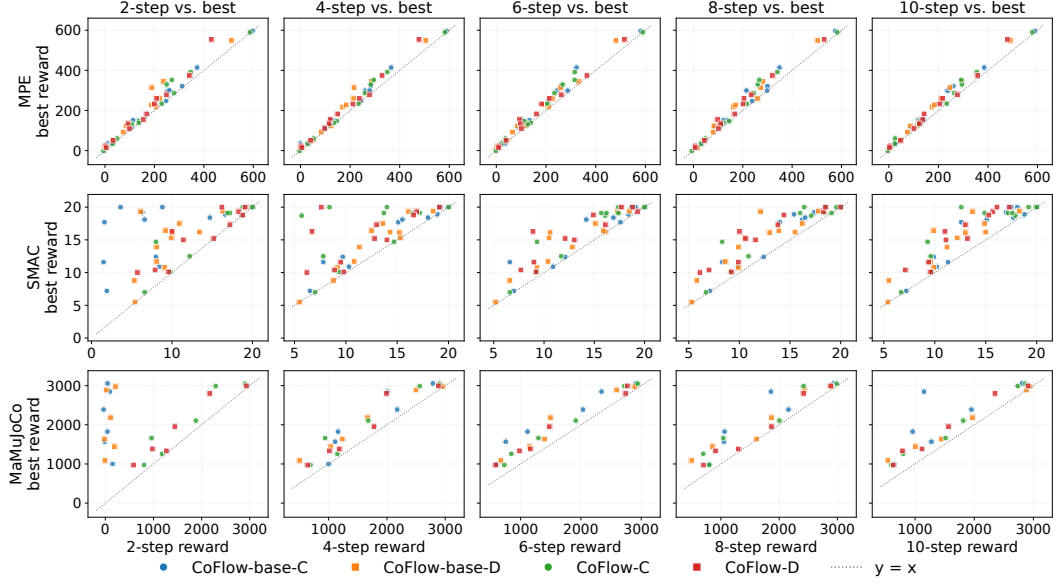


Figure 12: Companion to Figure 6 for even denoising budgets $k \in \{2, 4, 6, 8, 10\}$. Axes and legend as in the main-text figure.

A.6 Additional Coordination Analysis

Per-layer learned coordination gating. Figure 14 reports the learned γ_l scale for each CVA layer on five representative MPE configurations. The values back the main-text claim that γ_l flips sign across tasks and depth: on the cooperative coverage tasks Spread Expert and Spread Medium, the early layers $l=0, 1$ adopt positive gating while $l=2$ becomes negative; on the predator-prey pursuit tasks Tag Expert and Tag Medium the pattern inverts, with $l=0, 1$ negative and $l=2$ positive. World, which mixes coverage and pursuit with obstacles, settles on a uniformly negative early-layer profile. All active CVA layers satisfy $|\gamma_l| \leq 0.15$, while the deepest layer remains at $\gamma_l = 0$ from initialization. This is consistent with the Adaptive Coordination Gating regime $\bar{\sigma} \ll 1$ in Theorem B.2.

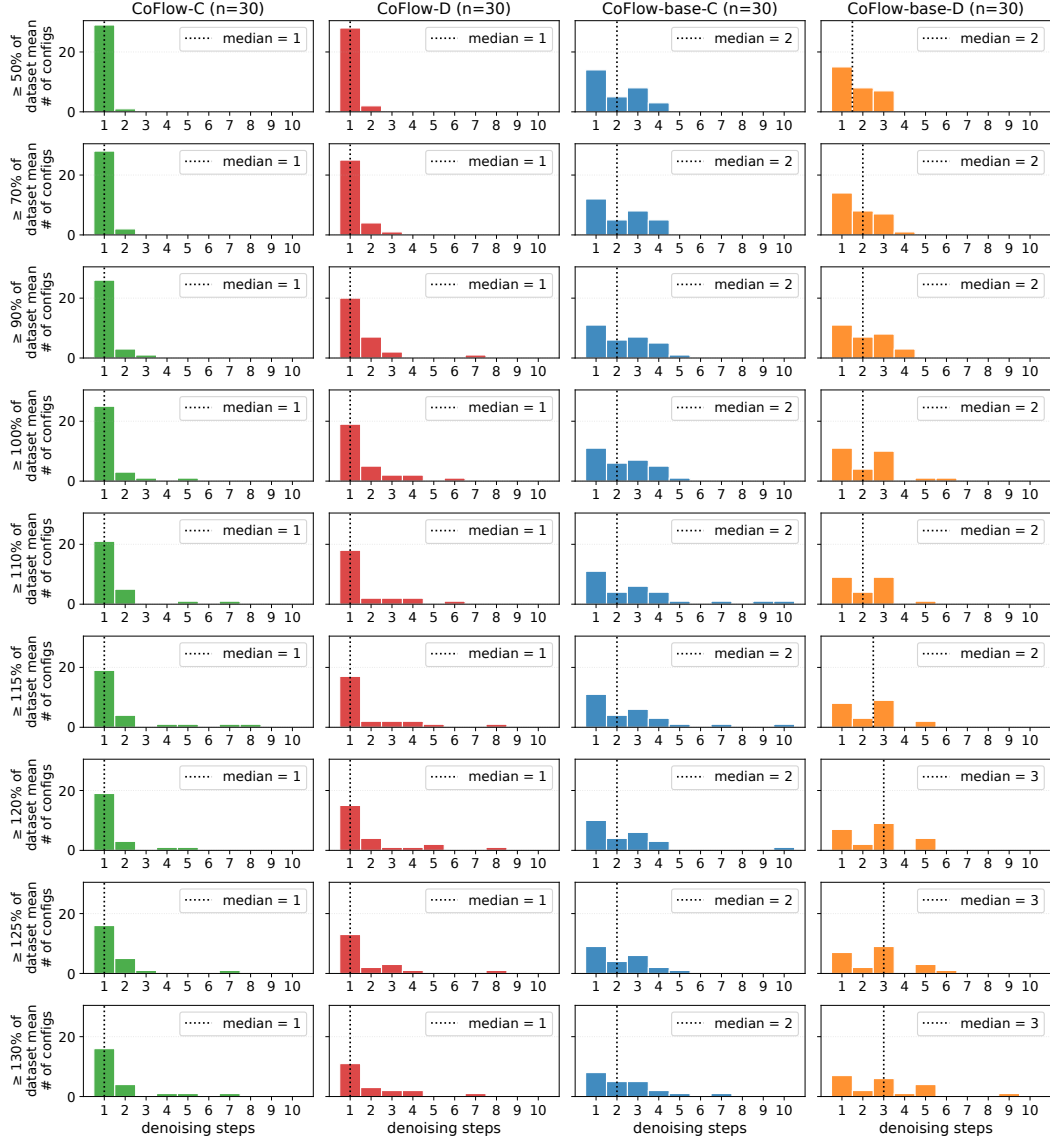


Figure 13: Steps needed to reach $\geq N\%$ of the offline dataset mean. Rows sweep the threshold over $N \in \{50, 70, 90, 100, 110, 115, 120, 125, 130\}$; columns are the four CoFlow variants CoFlow-C, CoFlow-D, CoFlow-base-C, and CoFlow-base-D. Dashed vertical lines mark the per-panel medians. Configurations whose reward never reaches the threshold within 10 steps are not shown, so the per-column n in the title shrinks as N grows.

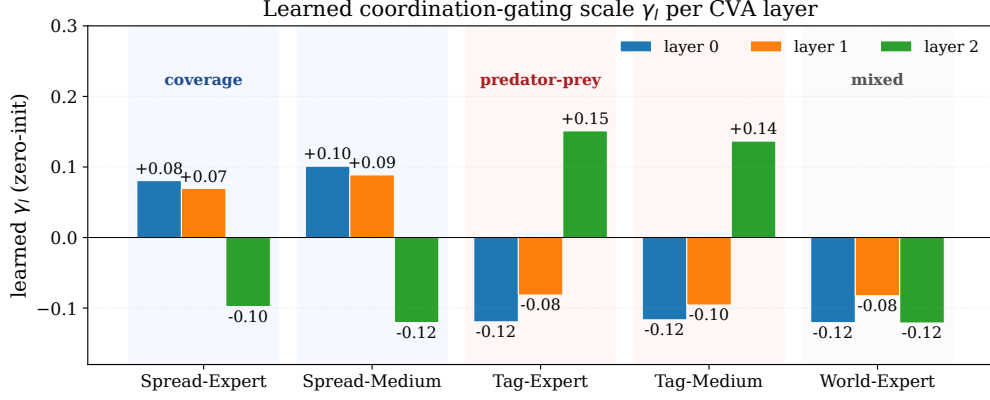


Figure 14: Learned γ_l per CVA layer on five MPE configurations, all under the centralized variant CoFlow-C. Background shading marks task type: blue for the coverage task Spread, red for predator-prey pursuit Tag, and gray for the mixed-environment task World. Coverage tasks favour positive early-layer gating, while predator-prey pursuit inverts the early-layer sign.

B Theoretical Foundations

This appendix collects the derivations, proofs, and algorithms that support the main text. §B.1 recaps the prerequisites: linear flow matching, the averaged velocity field with its consistency target, and the Wasserstein-2 distance. §B.2 fixes the notation. §B.3 shows why the finite-difference surrogate of Equation (6) is a controlled substitute for the JVP-based correction. §B.4 proves the joint velocity decomposition that bounds the cross-agent correction by directly measurable architectural quantities. §B.5 propagates the resulting training error to single-pass sample quality. §B.6 summarizes the team-size implication of the bound. §B.7 closes with implementation notes, and §B.8 gives the training and inference pseudocode.

B.1 Preliminaries

Three concepts underlie the proofs that follow: linear flow matching, the averaged velocity field with its consistency target, and the Wasserstein-2 distance. We summarize them briefly so that the proof statements stand on their own.

Linear flow matching. Given a clean joint trajectory $\tau_0 \sim p_\tau$ from the offline dataset and Gaussian noise $z_1 \sim \mathcal{N}(0, I)$, the linear interpolant

$$z_t = (1 - t) \tau_0 + t z_1, \quad t \in [0, 1], \quad (10)$$

defines a flow whose sample-conditional velocity is the constant $v_{\text{cond}} := z_1 - \tau_0$. Flow matching learns a velocity field $v_\theta(z, t)$ by least-squares regression against v_{cond} [Lipman et al., 2023, Liu et al., 2023].

Averaged velocity consistency. We learn an *averaged* velocity field

$$u_\theta(z_t, r, t) \approx \frac{1}{t - r} \int_r^t v^*(z_s, s) ds, \quad (11)$$

which integrates the instantaneous velocity over the interval $[r, t]$ rather than evaluating it at a single time point. For $r = 0$ and $t = 1$, the averaged velocity supports the single-pass shortcut $\hat{\tau}_0 = z_1 - u_\theta(z_1, 0, 1)$. To make this single-pass estimate accurate, averaged-velocity training imposes a consistency relation between $u_\theta(z_t, r, t)$ and $u_\theta(z_t, 0, t)$ that is usually implemented with a Jacobian-vector product (JVP) through the network. Such JVP-based training is memory-prohibitive for multi-agent backbones. Equation (6) replaces it with a damped finite-difference surrogate, and §B.3 gives the resulting Taylor control. When $r = t$, the averaged velocity is understood by its continuous extension to the instantaneous velocity limit.

Wasserstein-2 distance. For two probability measures μ, ν on $\mathbb{R}^{Nd}$ with finite second moment, the Wasserstein-2 distance is

$$W_2(\mu, \nu) := \left(\inf_{\pi \in \Pi(\mu, \nu)} \int \|x - y\|^2 d\pi(x, y) \right)^{1/2}, \quad (12)$$

where $\Pi(\mu, \nu)$ is the set of couplings of μ and ν . Two facts are used repeatedly. First, picking any specific coupling π gives an upper bound; in particular, when both samples can be expressed as functions of a common random variable ξ (an "identity coupling"),

$$W_2(\mu, \nu) \leq (\mathbb{E}_\xi \|F(\xi) - G(\xi)\|^2)^{1/2}. \quad (13)$$

Second, W_2 satisfies the triangle inequality $W_2(\mu, \nu) \leq W_2(\mu, \rho) + W_2(\rho, \nu)$, which we use to split the single-pass error into approximation and regression-gap terms.

B.2 Notation

All symbols used throughout the appendix are listed in the tables below. Unless stated otherwise, vector norms are Euclidean, matrix norms are Frobenius for feature tensors and spectral for linear maps, and p_X denotes the law of a random variable X .

Table 7: Joint trajectories, flow, and velocity-field symbols.

Symbol	Meaning
N	number of agents
d	flattened per-agent trajectory/state dimension used by the flow
$z_t \in \mathbb{R}^{Nd}$	joint state at flow-time $t \in [0, 1]$
$z_t^i \in \mathbb{R}^d$	agent- i component of z_t
r, t	reference and endpoint flow times, usually sampled with $0 \leq r \leq t \leq 1$
$z_1 \sim \mathcal{N}(0, I)$	noise endpoint
$\tau_0 \sim p_\tau$	data-side endpoint, drawn from data distribution p_τ
$v^*(z, t)$	true instantaneous velocity field along the flow
$v_{\text{cond}} := z_1 - \tau_0$	conditional regression target
$u_\theta(z_t, r, t)$	trained averaged velocity field over $[r, t]$
$V_\theta(z_t, r, t)$	finite-difference surrogate velocity used for training
$u^*(z_1, 0, 1) := \mathbb{E}[z_1 - \tau_0 \mid z_1]$	conditional-expectation oracle
$\hat{\tau}_0 := z_1 - u_\theta(z_1, 0, 1)$	model one-step estimate
$\tau_0^* := z_1 - u^*(z_1, 0, 1)$	oracle one-step estimate

Table 8: CVA layer quantities. The U-Net carries L Coordinated Velocity Attention layers indexed by $l \in \{1, \dots, L\}$.

Symbol	Meaning
L	number of CVA layers
$c_l^i \in \mathbb{R}^{T_l \times F_l}$	skip features of agent i at layer l
T_l, F_l	skip sequence length and per-token feature width
c_0^i	features entering the first CVA layer
$\alpha_l^{ij} \in [0, 1]$	softmax attention weights, $\sum_j \alpha_l^{ij} = 1$
$W_{V,l}$	learned value-projection matrix
$\sigma_{\max}(W_{V,l})$	largest singular value of $W_{V,l}$
$\gamma_l \in \mathbb{R}$	scalar gating coefficient, zero-initialized
$D_l := \max_{i \neq j} \ c_l^i - c_l^j\ $	pairwise feature diversity at layer l
$\bar{\sigma} := \max_l \gamma_l \sigma_{\max}(W_{V,l})$	gating-projection scale

Table 9: Per-agent decomposition and operators.

Symbol	Meaning
$\mathbf{e}_i \in \mathbb{R}^N$	i -th standard basis vector
$\mathbf{e}_i \otimes \bar{u}_\theta^i(z_t^i, t) \in \mathbb{R}^{Nd}$	block-stacking of the per-agent velocity into the i -th block
$\bar{u}_\theta^i(z_t^i, t) \in \mathbb{R}^d$	per-agent independent velocity (no-attention output for agent i)
$\Delta_{\text{attn}}(z_t, t) \in \mathbb{R}^{Nd}$	cross-agent correction (everything beyond the per-agent term)
I	identity matrix (distinct from the inverse-dynamics net I_ϕ^i)
$\text{sg}[\cdot] \equiv \text{stopgrad}[\cdot]$	stop-gradient operator
$W_2(\cdot, \cdot)$	Wasserstein-2 distance

Bound-specific quantities such as ϵ_{train} , ϵ_i , ϵ_{coord} , and κ_{reg} are introduced at the theorem where they are first used.

B.3 Taylor control of the finite-difference surrogate

This subsection justifies the training shortcut used in Equation (6). The exact averaged-velocity consistency update would use a time derivative of the averaged velocity field, which in practice means a Jacobian-vector product and second-order backpropagation through a large multi-agent backbone. CoFlow instead evaluates the same network twice, at the reference time r and endpoint time t , and inserts their difference as a detached correction. The question addressed here is therefore deliberately weaker than “did we compute the exact JVP?”: we need to show that the detached finite difference is a controlled local proxy whose effect vanishes with the time gap.

The point of the proof is local and practical: the two forward passes should move in the same leading Taylor direction as the derivative term that a JVP would expose, and the detached correction should become negligible when r and t are close. This is exactly what Proposition B.1 establishes.

Proposition B.1 (Taylor control for the damped FD surrogate). *Fix $z \in \mathbb{R}^{Nd}$ and $0 \leq r \leq t \leq 1$, and write $h := t - r$. Assume $u_\theta(z, 0, \cdot) \in C^2([0, 1])$, and let*

$$M_1(z) := \sup_{s \in [0, 1]} \|\partial_t u_\theta(z, 0, s)\|, \quad M_2(z) := \sup_{s \in [0, 1]} \|\partial_t^2 u_\theta(z, 0, s)\|. \quad (14)$$

The quantities M_1 and M_2 measure how quickly the velocity network changes along the time input while z is fixed. They are the only smoothness quantities needed to control the detached correction. First, the difference of two ordinary forward evaluations admits the Taylor expansion

$$u_\theta(z, 0, t) - u_\theta(z, 0, r) = h \partial_t u_\theta(z, 0, r) + R(z, r, t), \quad (15)$$

where the remainder is second order in the time gap:

$$\|R(z, r, t)\| \leq \frac{1}{2} h^2 M_2(z). \quad (16)$$

This is the step that links the practical two-forward implementation to the derivative direction that a JVP would expose. Substituting Equation (15) into Equation (6) then gives the actual surrogate used by CoFlow:

$$V_\theta(z, r, t) = u_\theta(z, 0, r) + \text{sg}[h^2 \partial_t u_\theta(z, 0, r) + h R(z, r, t)], \quad (17)$$

so the extra training signal is a detached, damped correction around the reference-time velocity. Its size obeys

$$\|V_\theta(z, r, t) - u_\theta(z, 0, r)\| \leq h^2 M_1(z) + \frac{1}{2} |h|^3 M_2(z). \quad (18)$$

Thus the FD surrogate applies a bounded stop-gradient correction: it nudges $u_\theta(z, 0, r)$ toward the endpoint-time velocity, but its influence shrinks quadratically or faster as t approaches r . This is the property needed for memory-efficient consistency training; no JVP or double-backward computation is required.

Proof. We first isolate the one-dimensional time argument, because the claim concerns only the variation of the network output between the two queried times. Let $f(s) := u_\theta(z, 0, s)$, which lies in $C^2([0, 1])$ by assumption. Taylor’s theorem with integral remainder gives, for any $r, t \in [0, 1]$,

$$f(t) = f(r) + (t - r) f'(r) + \int_r^t (t - s) f''(s) ds. \quad (19)$$

Subtracting $f(r)$ from both sides is exactly the finite difference computed by the two forward passes, so it yields the identity in Equation (15) with

$$R(z, r, t) = \int_r^t (t - s) f''(s) ds. \quad (20)$$

The remaining task is to show that this substitution does not introduce an uncontrolled term. Using the definition of $M_2(z)$,

$$\|R(z, r, t)\| \leq \int_{\min(r, t)}^{\max(r, t)} |t - s| \|f''(s)\| ds \leq M_2(z) \int_{\min(r, t)}^{\max(r, t)} |t - s| ds = \frac{1}{2} h^2 M_2(z). \quad (21)$$

Substituting the Taylor decomposition into the damped stop-gradient term of $V_\theta = u_\theta(z, 0, r) + h \text{sg}[u_\theta(z, 0, t) - u_\theta(z, 0, r)]$ produces the displayed surrogate relation. The norm bound follows from the triangle inequality and the definition of $M_1(z)$, completing the proof. $\square$

The proposition is stated per sample because that is where the surrogate’s role is clearest. The training objective averages the same correction over trajectories, noise, and sampled time pairs; as long as the sampled time gaps have finite low-order moments, this averaging preserves the same qualitative conclusion. In particular, the detached part remains a time-gap-controlled perturbation of reference-time regression rather than a different learning target. The algorithm uses logit-normal time pairs restricted to $r \leq t$; see Algorithm 1, line 4.

B.4 Joint Velocity Decomposition

We turn to the architectural side. The CVA backbone produces a joint averaged velocity $u_\theta(z_t, 0, t)$ that mixes per-agent dynamics with cross-agent coordination. We decompose the joint output into a per-agent term plus a cross-agent correction, and bound the correction by two architectural quantities that can be read off the trained model: the gating–projection scale $\bar{\sigma}$ and the inter-agent feature diversity D_0 . This should be read as a controllability statement: coordination can be nonzero, but its magnitude is limited by the learned gates and by how different the agents’ features are. For readability, the bound below treats the non-attention U-Net blocks between CVA layers as non-expansive after normalization; if these blocks have Lipschitz constants larger than one, their product multiplies the right-hand side without changing the dependence on N , L , $\bar{\sigma}$, or D_0 .

Theorem B.2 (Joint Velocity Decomposition). *The joint averaged velocity field u_θ produced by a network with L CVA layers admits the decomposition*

$$u_\theta(z_t, 0, t) = \sum_{i=1}^N \mathbf{e}_i \otimes \bar{u}_\theta^i(z_t^i, t) + \Delta_{\text{attn}}(z_t, t), \quad (22)$$

with the cross-agent correction bounded by

$$\|\Delta_{\text{attn}}\| \leq \frac{\sqrt{N}}{3} [(1 + 3\bar{\sigma})^L - 1] D_0. \quad (23)$$

In the small-gating regime $\bar{\sigma} \ll 1$ the bound simplifies to

$$\|\Delta_{\text{attn}}\| \lesssim \sqrt{N} L \bar{\sigma} D_0. \quad (24)$$

Equivalently, the per-agent normalized norm $N^{-1/2} \|\Delta_{\text{attn}}\|$ removes the leading $\sqrt{N}$ factor.

Proof. We argue by induction over the L CVA layers. Define the per-layer gating–projection scale and its uniform upper bound

$$\bar{\sigma}_l := |\gamma_l| \sigma_{\max}(W_{V,l}), \quad \bar{\sigma} := \max_l \bar{\sigma}_l. \quad (25)$$

Base case ($L = 0$, **no attention**). Without cross-agent attention, the network processes each agent independently through the shared backbone, so the output for agent i depends only on z_t^i and t :

$$u_\theta(z_t, 0, t)|_{\text{agent } i} = \bar{u}_\theta^i(z_t^i, t), \quad (26)$$

and $\Delta_{\text{attn}} = 0$. The bound holds trivially.

Inductive step. We track two quantities through the layers: the interaction perturbation δ_l^i that captures the component of c_l^i due to cross-agent coupling, and the pairwise feature diversity D_l .

Inductive hypothesis. After $l - 1$ layers, the feature for agent i decomposes as

$$c_{l-1}^i = \bar{c}_{l-1}^i + \delta_{l-1}^i, \quad (27)$$

where $\bar{c}_{l-1}^i$ is the no-attention component that depends only on agent i ’s own input, and the diversity satisfies $D_{l-1} \leq (1 + 3\bar{\sigma})^{l-1} D_0$.

At layer l , cross-agent attention with residual connection produces

$$c_l^i = c_{l-1}^i + \gamma_l \sum_{j=1}^N \alpha_l^{ij} W_{V,l} c_{l-1}^j \quad (28)$$

$$= \underbrace{(I + \gamma_l W_{V,l}) c_{l-1}^i}_{\text{self term}} + \underbrace{\gamma_l \sum_{j=1}^N \alpha_l^{ij} W_{V,l} (c_{l-1}^j - c_{l-1}^i)}_{\text{cross-agent interaction}}, \quad (29)$$

where the second line uses the softmax normalization $\sum_j \alpha_l^{ij} = 1$.

Bounding the interaction at layer l . Since $\alpha_l^{ij} \geq 0$ and $\sum_j \alpha_l^{ij} = 1$,

$$\left\| \gamma_l \sum_j \alpha_l^{ij} W_{V,l} (c_{l-1}^j - c_{l-1}^i) \right\| \leq \bar{\sigma}_l \cdot D_{l-1}. \quad (30)$$

692 *Propagation of diversity.* For any pair (i, j) ,

$$\|c_l^i - c_l^j\| \leq \|(I + \gamma_l W_{V,l})(c_{l-1}^i - c_{l-1}^j)\| + 2\bar{\sigma}_l D_{l-1} \quad (31)$$

$$\leq (1 + \bar{\sigma}_l)D_{l-1} + 2\bar{\sigma}_l D_{l-1} = (1 + 3\bar{\sigma}_l)D_{l-1}. \quad (32)$$

693 Iterating this recurrence with $\bar{\sigma}_l \leq \bar{\sigma}$ uniformly gives

$$D_l \leq (1 + 3\bar{\sigma})^l D_0. \quad (33)$$

694 *Accumulating the total interaction.* For each layer, the per-agent interaction has norm at most
 695 $\bar{\sigma}_l D_{l-1}$, so the corresponding joint vector over N agents has Euclidean norm at most $\sqrt{N}\bar{\sigma}_l D_{l-1}$.
 696 The total attention perturbation Δ_{attn} is the sum of interaction terms across all L layers. Using
 697 $D_{l-1} \leq (1 + 3\bar{\sigma})^{l-1} D_0$,

$$\|\Delta_{\text{attn}}\| \leq \sqrt{N} \sum_{l=1}^L \bar{\sigma}_l \cdot D_{l-1} \leq \sqrt{N}\bar{\sigma} \sum_{l=1}^L (1 + 3\bar{\sigma})^{l-1} D_0 \quad (34)$$

$$= \sqrt{N}\bar{\sigma} \cdot \frac{(1 + 3\bar{\sigma})^L - 1}{3\bar{\sigma}} \cdot D_0 = \frac{\sqrt{N}}{3} [(1 + 3\bar{\sigma})^L - 1] D_0. \quad (35)$$

698 *Small- $\bar{\sigma}$ regime.* When the zero-initialized gates remain in the empirically observed small-gating
 699 regime, $\bar{\sigma} \ll 1$. Using $(1 + 3\bar{\sigma})^L \leq e^{3L\bar{\sigma}} \approx 1 + 3L\bar{\sigma}$ for small $\bar{\sigma}$,

$$\|\Delta_{\text{attn}}\| \lesssim \sqrt{N} L \bar{\sigma} D_0. \quad (36)$$

700 **Vanishing of the cross-agent correction.** When $c_0^i = c_0^j$ for all i, j , we have $D_0 = 0$, so $\Delta_{\text{attn}} = 0$
 701 by the bound above. The converse implication is not needed for the guarantee: degenerate gates,
 702 value projections, or feature directions can also suppress the correction. Thus D_0 should be read as
 703 an upper-envelope quantity, not as an if-and-only-if certificate of coordination. $\square$

704 B.5 Single-Pass Wasserstein Error Bound

705 Theorem B.2 bounds the architectural correction Δ_{attn} , but it does not yet say anything about sample
 706 quality. We now propagate the training error through to the distribution $p_{\hat{\tau}_0}$ produced by single-pass
 707 inference, measured in Wasserstein-2 distance against the data distribution p_τ . The bound has two
 708 parts: an approximation term controlled by the trained network and a regression-gap term for the
 709 oracle one-step estimator. Thus only the first term is directly reduced by better training; the second
 710 term reflects the intrinsic limitation of a deterministic conditional-mean shortcut.

711 Let p_τ denote the true data distribution over joint trajectories $\tau \in \mathbb{R}^{Nd}$.

712 **Assumption B.3** (Network approximation). The trained network u_θ achieves an ϵ_{train} -approximation
 713 of the conditional expectation of the averaged velocity field:

$$\mathbb{E}_{z_1 \sim \mathcal{N}(0, I)} \|u_\theta(z_1, 0, 1) - u^*(z_1, 0, 1)\|^2 \leq \epsilon_{\text{train}}^2, \quad (37)$$

714 where $u^*(z_1, 0, 1) := \mathbb{E}_{\tau_0 | z_1} [z_1 - \tau_0]$ is the regression target, i.e. the conditional mean of $z_1 - \tau_0$
 715 under the source-data coupling used to define the stochastic interpolant.

716 Observe that u^* is the conditional expectation of the velocity given z_1 , not the sample-specific
 717 velocity. When $p(\tau_0 | z_1)$ is not a point mass, the oracle one-step estimate $\tau_0^* := z_1 - u^*(z_1, 0, 1)$
 718 recovers the conditional mean instead of the exact sample, so $W_2(p_{\tau_0^*}, p_\tau) > 0$ in general.

719 **Theorem B.4** (Single-pass Wasserstein error bound). *Under Assumption B.3, the single-pass sample*
 720 *distribution $p_{\hat{\tau}_0}$ satisfies*

$$W_2(p_{\hat{\tau}_0}, p_\tau) \leq \epsilon_{\text{train}} + \kappa_{\text{reg}}, \quad (38)$$

721 where $\kappa_{\text{reg}} := W_2(p_{\tau_0^*}, p_\tau)$ is the regression gap of the oracle one-step estimator. To decompose the
 722 training error, choose an analogous per-agent-plus-residual decomposition for the oracle u^* ,

$$u^*(z_1, 0, 1) = \sum_{i=1}^N \mathbf{e}_i \otimes \bar{u}^{i,*}(z_1^i, 1) + \Delta_{\text{attn}}^*(z_1, 1), \quad (39)$$

723 and define

$$\epsilon_i^2 := \mathbb{E} \|\bar{u}_\theta^i - \bar{u}^{i,*}\|^2, \quad (40)$$

$$\epsilon_{\text{coord}}^2 := \mathbb{E} \|\Delta_{\text{attn}} - \Delta_{\text{attn}}^*\|^2. \quad (41)$$

724 Here and below, the shared arguments $(z_1, 0, 1)$ are suppressed inside these error terms. Then the
 725 training error itself decomposes as

$$\epsilon_{\text{train}} \leq \sqrt{\sum_{i=1}^N \epsilon_i^2} + \epsilon_{\text{coord}}, \quad (42)$$

726 *Proof.* The proof proceeds in four steps: triangle decomposition, approximation error, regression
 727 gap, and training error decomposition.

728 **Step 1: Wasserstein decomposition.** Let

$$\hat{\tau}_0 = z_1 - u_\theta(z_1, 0, 1), \quad \tau_0^* = z_1 - u^*(z_1, 0, 1) \quad (43)$$

729 denote the one-step estimate and the oracle one-step estimate. By the triangle inequality on W_2 ,

$$W_2(p_{\hat{\tau}_0}, p_\tau) \leq \underbrace{W_2(p_{\hat{\tau}_0}, p_{\tau_0^*})}_{\text{approximation error}} + \underbrace{W_2(p_{\tau_0^*}, p_\tau)}_{\text{regression gap}}. \quad (44)$$

730 **Step 2: Approximation error.** Using the identity coupling via the shared z_1 and the coupling
 731 definition of W_2 ,

$$W_2(p_{\hat{\tau}_0}, p_{\tau_0^*}) \leq (\mathbb{E} \|\hat{\tau}_0 - \tau_0^*\|^2)^{1/2} \quad (45)$$

$$= (\mathbb{E} \|u_\theta(z_1, 0, 1) - u^*(z_1, 0, 1)\|^2)^{1/2} \leq \epsilon_{\text{train}}. \quad (46)$$

732 **Step 3: Regression gap.** Define

$$\kappa_{\text{reg}} := W_2(p_{\tau_0^*}, p_\tau). \quad (47)$$

733 This term is unavoidable for any one-step estimator that maps the source noise through the conditional
 734 mean velocity u^* . If the source–data coupling leaves multiple plausible data trajectories for the same
 735 z_1 , the deterministic shortcut returns their conditional mean rather than a particular sample. The
 736 triangle decomposition above therefore gives

$$W_2(p_{\hat{\tau}_0}, p_\tau) \leq \epsilon_{\text{train}} + \kappa_{\text{reg}}. \quad (48)$$

737 **Step 4: Training error decomposition.** Decompose u_θ as in Theorem B.2 and use the oracle
 738 decomposition from the theorem statement. Since the N agent blocks indexed by $\mathbf{e}_i$ are orthogonal,

$$\epsilon_{\text{train}}^2 = \mathbb{E} \left\| \sum_i \mathbf{e}_i \otimes (\bar{u}_\theta^i - \bar{u}^{i,*}) + (\Delta_{\text{attn}} - \Delta_{\text{attn}}^*) \right\|^2 \quad (49)$$

$$= \underbrace{\sum_i \epsilon_i^2}_{\text{per-agent}} + 2 \underbrace{\mathbb{E} \left\langle \sum_i \mathbf{e}_i \otimes (\bar{u}_\theta^i - \bar{u}^{i,*}), \Delta_{\text{attn}} - \Delta_{\text{attn}}^* \right\rangle}_{\text{cross-term}} + \underbrace{\epsilon_{\text{coord}}^2}_{\text{coordination}}. \quad (50)$$

739 By Cauchy–Schwarz, the cross-term satisfies $|\text{cross-term}| \leq \sqrt{\sum_i \epsilon_i^2} \epsilon_{\text{coord}}$. Therefore

$$\epsilon_{\text{train}}^2 \leq \sum_i \epsilon_i^2 + 2 \sqrt{\sum_i \epsilon_i^2} \epsilon_{\text{coord}} + \epsilon_{\text{coord}}^2 = \left(\sqrt{\sum_i \epsilon_i^2} + \epsilon_{\text{coord}} \right)^2, \quad (51)$$

740 giving $\epsilon_{\text{train}} \leq \sqrt{\sum_i \epsilon_i^2} + \epsilon_{\text{coord}}$. $\square$

741 B.6 Scaling with the number of agents

742 The training-error decomposition of Theorem B.4 already contains the main team-size message. If
 743 the per-agent errors are uniformly bounded by $\bar{\epsilon}$, then the independent part contributes at most $\sqrt{N} \bar{\epsilon}$
 744 in the joint Euclidean norm. The coordination part inherits the attention envelope in Theorem B.2:
 745 with $D_N := \max_{i \neq j} \|c_0^i - c_0^j\|$ and a relative coordination approximation factor ϵ_{attn} , the combined
 746 implication is

$$\epsilon_{\text{train}} \lesssim \sqrt{N} \bar{\epsilon} + \sqrt{N} L \bar{\sigma} D_N \epsilon_{\text{attn}} \quad (\bar{\sigma} \ll 1). \quad (52)$$

747 The leading $\sqrt{N}$ factor is a consequence of measuring a joint vector in Euclidean norm; it disappears
 748 after per-agent normalization. Thus the quantity that matters for coordination scalability is not team
 749 size alone, but whether feature diversity D_N and the learned gate scale $\bar{\sigma}$ remain controlled as the
 750 team grows.

B.7 Implementation Notes

We collect implementation-level remarks that are referenced from §3 but are not required for the proofs above.

Averaged velocity field. Along a flow trajectory, the instantaneous velocity field $v(z_t, t)$ and the averaged velocity field $u(z_t, r, t)$ are related by

$$v(z_t, t) = u(z_t, 0, t) + t \frac{d}{dt} u(z_t, 0, t), \quad (53)$$

where the derivative is the total derivative along the path and includes the dependence of u on z_t . Learning u is sufficient to recover v in principle, while CoFlow’s training objective uses the finite-difference surrogate directly to avoid computing this derivative.

CVA skip connections. At encoder layer l , each agent i has skip features $c_l^i \in \mathbb{R}^{T_l \times F_l}$. The CVA-enhanced feature $\hat{c}_l^i$ is then concatenated with the original encoder feature e_l^i before entering the decoder:

$$\tilde{h}_l^i = \text{Concat}(e_l^i, \hat{c}_l^i). \quad (54)$$

CVA operates independently at each layer; it is layer-local but agent-global. The Adaptive Coordination Gate zero-initializes γ_l , so training begins from the independent-agent solution and only progressively activates coordination, which keeps the model inside the small- $\bar{\sigma}$ regime of Theorem B.2.

CTDE execution modes. During centralized training the model accesses joint observations $o_t = (o_t^1, \dots, o_t^N)$ and produces

$$\hat{\tau}_0 = z_1 - u_\theta(z_1, 0, 1 \mid o_t, R). \quad (55)$$

At decentralized deployment, agent i uses a masked joint context $m_i(o_t)$ that keeps its own local observation history and masks unavailable teammate observations:

$$\hat{\tau}_0^{(i)} = z_1 - u_\theta(z_1, 0, 1 \mid m_i(o_t), R). \quad (56)$$

Agent i then executes only its own block of the generated trajectory. The CVA mechanism learned during centralized training enables implicit teammate modeling without communication. The shared inverse-dynamics head finally maps predicted states to actions via $a_t^i = I_\phi^i(o_t^i, \hat{o}_{t+1}^i)$.

Return-conditioned velocity guidance. Classifier-free guidance interpolates between the unconditional and conditional velocity fields:

$$\hat{u}_\theta(z_t, 0, t) = u_\theta(z_t, 0, t \mid \emptyset) + \omega(u_\theta(z_t, 0, t \mid R) - u_\theta(z_t, 0, t \mid \emptyset)). \quad (57)$$

A larger ω biases toward higher-return trajectories at the cost of reduced diversity. During training the return condition is dropped with probability 0.25 to learn the unconditional model.

Temporal context augmentation. For locomotion tasks that require temporal awareness we condition on a sliding window of past observations,

$$h_t = [o_{t-C}, \dots, o_t], \quad (58)$$

of length C . This captures coordination dynamics such as gait synchronization. History conditioning effectively reduces the feature diversity term in the scaling discussion of Appendix B.6, since agents’ features become more predictable given past patterns; this matches the 28% MA-MuJoCo improvement reported in Table 5.

B.8 Training and Inference Algorithms

We provide the detailed training and inference algorithms for CoFlow.

CoFlow-base variant. The ablation variant *CoFlow-base* used throughout the step analyses is obtained from Algorithm 1 by removing the finite-difference surrogate term (lines 9–12) and regressing directly against the conditional velocity. Concretely, the surrogate loss $\mathcal{L}_{\text{vel}}$ is replaced with the standard averaged-velocity loss $\mathcal{L}_{\text{flow}}$ of Equation (2), that is $\|u_\theta(z_t, 0, t) - v_{\text{cond}}\|^2$. Both variants share the identical CVA-augmented backbone and inverse-dynamics head.

Remark on the surrogate. Proposition B.1 shows that the damped FD correction is Taylor-controlled by the first and second time derivatives of u_θ , so its objective-level influence is governed by the sampled time gap. We use this finite-difference form rather than a true `torch.autograd.functional.jvp` call to keep training memory compatible with the multi-agent backbone, and verify experimentally that the surrogate is sufficient to recover the few-step property observed in Figures 5, 10 and 11.

Algorithm 1 CoFlow Training (default variant)

Require: Offline dataset $\mathcal{D}$, number of agents N , velocity network u_θ , inverse-dynamics network I_ϕ , flow ratio ρ , learning rate η , inverse-dynamics weight λ

- 1: **repeat**
- 2: Sample batch of trajectories $\{\tau^i\}_{i=1}^N$ and returns R from $\mathcal{D}$; define $\tau_0 = (\tau^1, \dots, \tau^N)$
- 3: Sample noise $z_1 \sim \mathcal{N}(0, I)$
- 4: Sample time pairs (t, r) from logit-normal distribution with $r \leq t$
- 5: With probability ρ , set $r \leftarrow t$ for flow consistency
- 6: Compute interpolated trajectory: $z_t = (1 - t) \cdot \tau_0 + t \cdot z_1$
- 7: Compute conditional velocity: $v_{\text{cond}} = z_1 - \tau_0$
- 8: // Improved averaged-velocity loss (finite-difference surrogate of the averaged-velocity identity)
- 9: Evaluate at the two time queries: $u_t \leftarrow u_\theta(z_t, 0, t)$, $u_r \leftarrow u_\theta(z_t, 0, r)$
- 10: Stop-gradient finite difference: $\delta \leftarrow \text{stopgrad}(u_t - u_r)$
- 11: Compound velocity: $V \leftarrow u_r + (t - r) \delta$ // damped FD v-loss construction
- 12: $\mathcal{L}_{\text{vel}} = \|V - v_{\text{cond}}\|^2$
- 13: // Inverse dynamics loss (per-agent)
- 14: **for** each agent $i = 1, \dots, N$ **do**
- 15: Extract observations (o_t^i, o_{t+1}^i) and actions a_t^i from τ^i
- 16: $\mathcal{L}_{\text{act}}^i = \|a_t^i - I_\phi^i(o_t^i, o_{t+1}^i)\|^2$
- 17: **end for**
- 18: $\mathcal{L}_{\text{act}} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{\text{act}}^i$
- 19: $\mathcal{L} = \mathcal{L}_{\text{vel}} + \lambda \mathcal{L}_{\text{act}}$
- 20: Update parameters: $(\theta, \phi) \leftarrow (\theta, \phi) - \eta \nabla \mathcal{L}$
- 21: **until** convergence

Algorithm 2 CoFlow Inference (One-Step Generation)

Require: Trained velocity network u_θ , trained inverse-dynamics network I_ϕ , current observations $\{o_t^i\}_{i=1}^N$, target return R , guidance weight ω

- 1: Sample noise $z_1 \sim \mathcal{N}(0, I)$
- 2: // One-step generation using averaged velocity field
- 3: $\hat{\tau}_0 = z_1 - u_\theta(z_1, 0, 1 \mid \{o_t^i\}, R)$
- 4: // Optional: Classifier-free guidance
- 5: **if** $\omega > 1$ **then**
- 6: $u_{\text{cond}} = u_\theta(z_1, 0, 1 \mid \{o_t^i\}, R)$
- 7: $u_{\text{uncond}} = u_\theta(z_1, 0, 1 \mid \{o_t^i\}, \emptyset)$
- 8: $\hat{u} = u_{\text{uncond}} + \omega (u_{\text{cond}} - u_{\text{uncond}})$
- 9: $\hat{\tau}_0 = z_1 - \hat{u}$
- 10: **end if**
- 11: // Extract predicted next observations and compute actions
- 12: **for** each agent $i = 1, \dots, N$ **do**
- 13: Extract $\hat{o}_{t+1}^i$ from $\hat{\tau}_0$
- 14: $a_t^i = I_\phi^i(o_t^i, \hat{o}_{t+1}^i)$
- 15: **end for**
- 16: **return** Actions $\{a_t^i\}_{i=1}^N$

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and §3–§4 state and back the three contributions (CVA + Adaptive Coordination Gating with the finite-difference consistency surrogate, the structural decomposition framework, and the coordination-evidence methodology with 60-configuration evaluation). Limitations are explicitly acknowledged in §5.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: §5 discusses adaptive-step generation, integration of conservative Q-estimates, scaling beyond tens of agents, the worst-case nature of our error bounds, and generalization beyond CVA to communication-, graph-, or value-decomposition-based coordination architectures. Appendix A also documents the 5m_vs_6m-Good CoFlow-D decentralized gap and traces it to the decentralized attention mask.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The approximation assumption is stated in Appendix B (Assumption B.3); full proofs of Proposition B.1 and Theorems B.2 and B.4 are given in Appendix B, with cross-references to the assumptions used.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The architecture is fully described in §3 and Appendix B; hyperparameters are listed in Appendix A; datasets follow MADiff/off-the-grid public splits; we use 3 random seeds and report mean $\pm$ std for all reported numbers.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code and configurations will be released upon acceptance under an anonymized URL; an anonymized snapshot is included with the supplemental submission. All datasets and benchmarks (MPE/OMAR, MA-MuJoCo/off-the-grid, SMAC) are publicly available.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Training/inference protocols are described in §4; full hyperparameter tables, optimizer settings, and per-environment splits are in Appendix A.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All reported numbers are mean $\pm$ standard deviation across 3 random seeds. The factor of variability is the random seed (architecture, dataset, and hyperparameters fixed). Error bars in figures are 1-sigma standard error of the mean across seeds and configurations, as stated in their captions.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Training and evaluation use a single NVIDIA RTX 3090 (24 GB) per run; per- (env, task, quality) training takes 6–12 hours, and the 60-configuration sweep at evaluation takes roughly 2–3 GPU-days. Total project compute including extra debug configurations is approximately one order of magnitude larger than the reported runs.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work does not involve human subjects, personal data, or known dual-use risks. It uses only public benchmarks. We have reviewed and conform to the NeurIPS Code of Ethics.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Impact Statement notes that the work advances offline multi-agent decision making and that we are not aware of negative societal impacts that require specific mitigation in this foundational regime.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release pretrained models or scraped data with high misuse risk; it releases an offline MARL algorithm and configurations for public benchmarks.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cite MADiff, MADiTS, off-the-grid MARL, SMAC, MPE, and the OMAR dataset; we use them under their public licenses. Code dependencies (PyTorch, etc.) are used within their stated licenses.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce a new dataset; the released code (upon acceptance) will be documented with usage instructions, but it is not the primary contribution.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human subject research.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

906 Answer: [N/A]
907 Justification: The paper does not involve human subject research and therefore did not
908 require IRB review.
909 **16. Declaration of LLM usage**
910 Question: Does the paper describe the usage of LLMs if it is an important, original, or
911 non-standard component of the core methods in this research? Note that if the LLM is used
912 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
913 scientific rigor, or originality of the research, declaration is not required.
914 Answer: [N/A]
915 Justification: LLMs are not part of the core methodology, models, or empirical pipeline of
916 this work; any LLM use was limited to writing-assistance for the manuscript and does not
917 require declaration.